

Note: Initial upload on 12/9/2023 for pre-reading, but subject to minor changes.

Week 8 Graphs

Algorithms and Data Structures
COMP3506/7505

Week 8 – Graphs

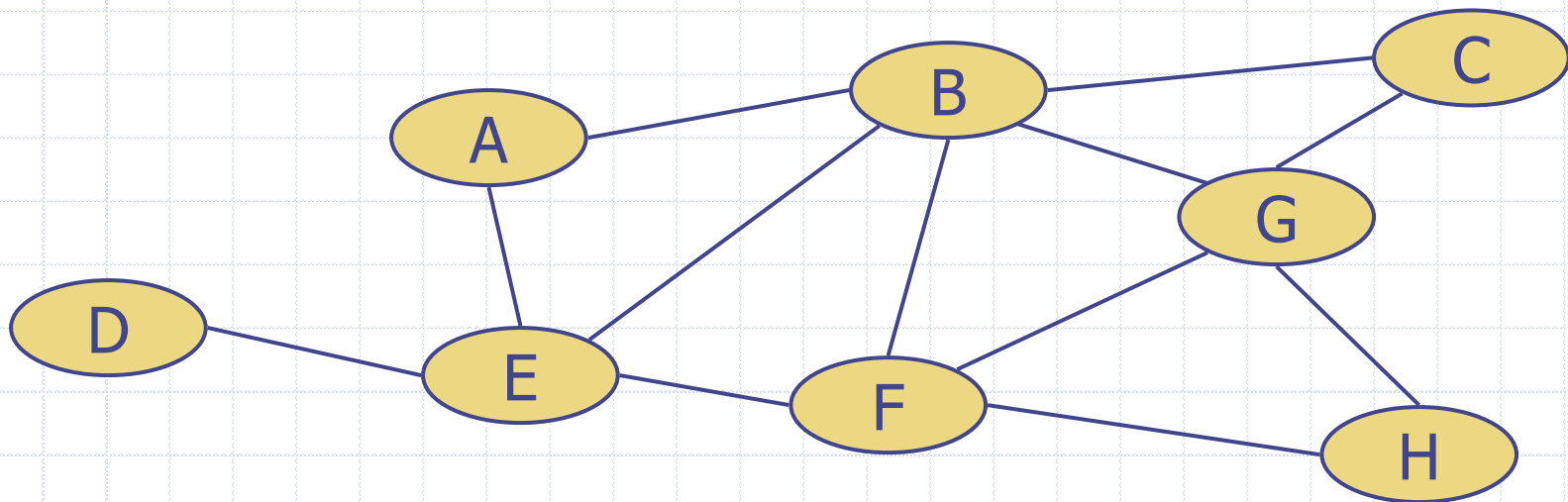
1. Graph fundamentals
2. Graph implementation
3. Depth-first search
4. Breadth-first search
5. Directed graphs

Week 8 – Graphs

1. Graph fundamentals
2. Graph implementation
3. Depth-first search
4. Breadth-first search
5. Directed graphs

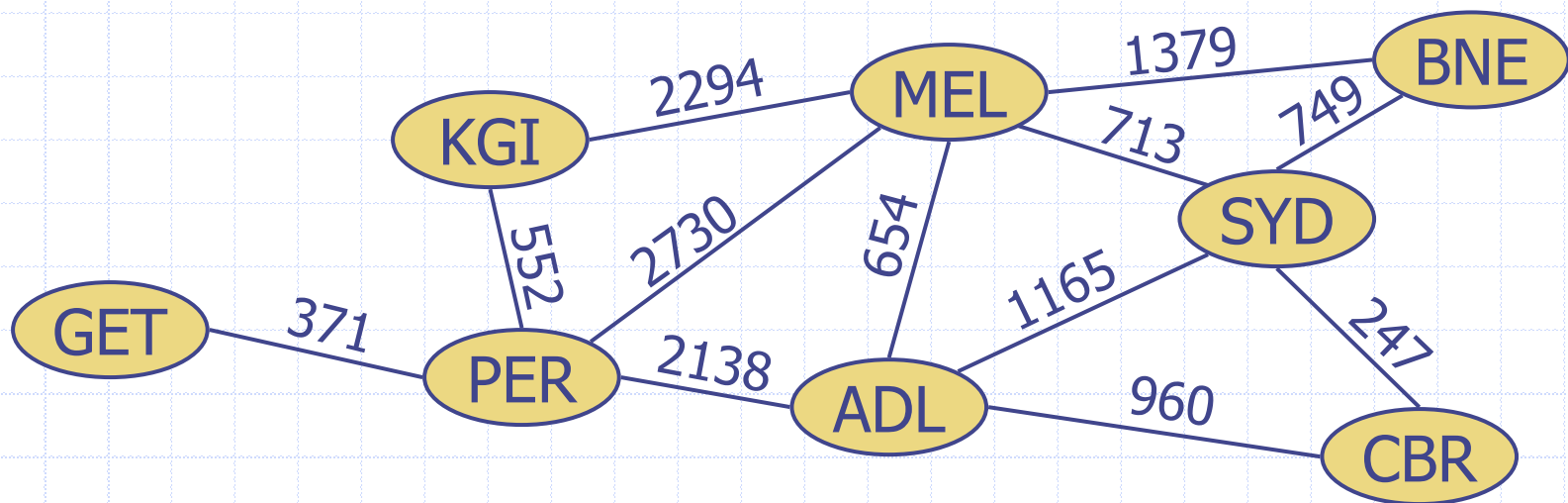
Graphs

- A graph is a pair (V, E) , where
 - V is a set of nodes, called **vertices**
 - E is a collection of pairs of vertices, called **edges**
 - Both vertices and edges can be thought of as positions and may store elements



Graph Example

- ❑ Vertex represents an airport and stores the airport code
- ❑ Edge represents a flight route between two airports and stores distance of the route



Edge Types

- Undirected edge
 - unordered pair of vertices (u,v)
 - e.g. a flight route
- Undirected graph
 - all the edges are undirected
 - e.g. flight route network



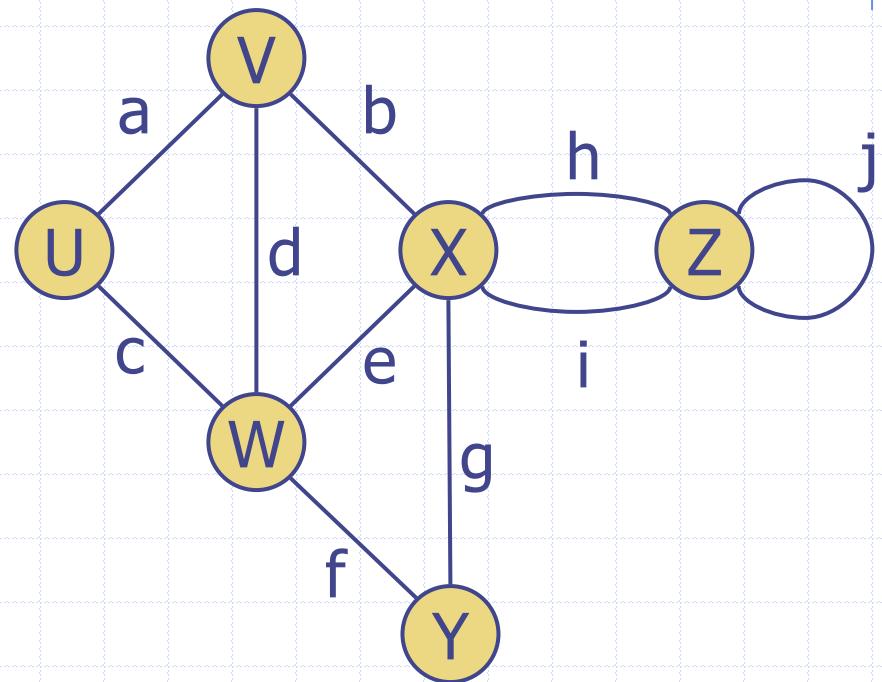
Edge Types

- Directed edge
 - ordered pair of vertices (u,v)
 - first vertex u is the origin
 - second vertex v is the destination
 - e.g. a flight
- Directed graph
 - all the edges are directed
 - e.g. flight network



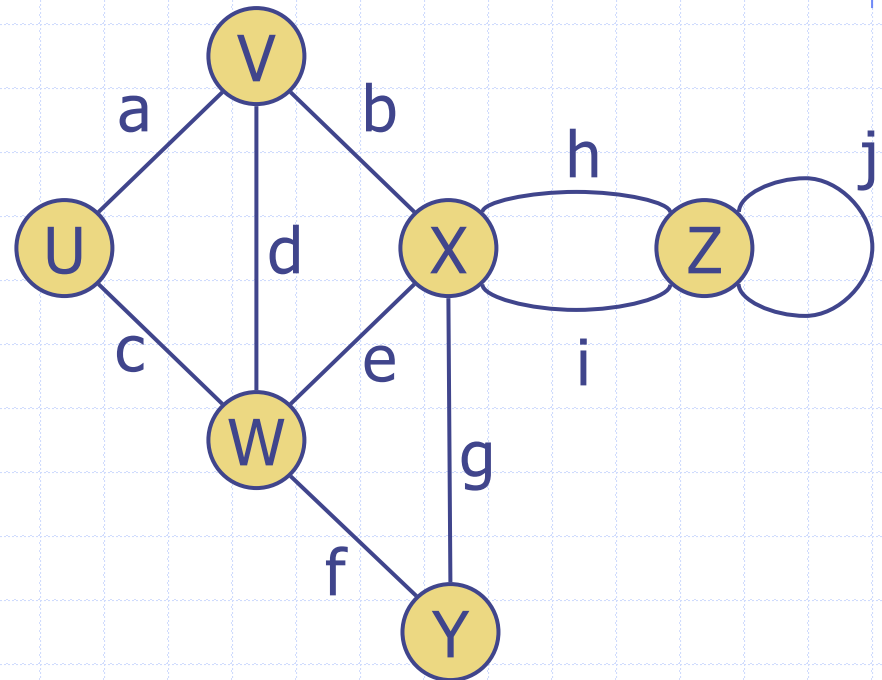
Terminology

- End vertices (or endpoints) of an edge
 - U and V are endpoints of a
- Edges incident on a vertex
 - a, d, and b are incident on V
- Adjacent vertices
 - U and V are adjacent
 - ◆ connected by a vertex



Terminology (cont.)

- Degree of a vertex
 - X has degree 5
- Parallel edges
 - h and i are parallel edges
- Self-loop
 - j is a self-loop



Terminology (cont.)

□ Path

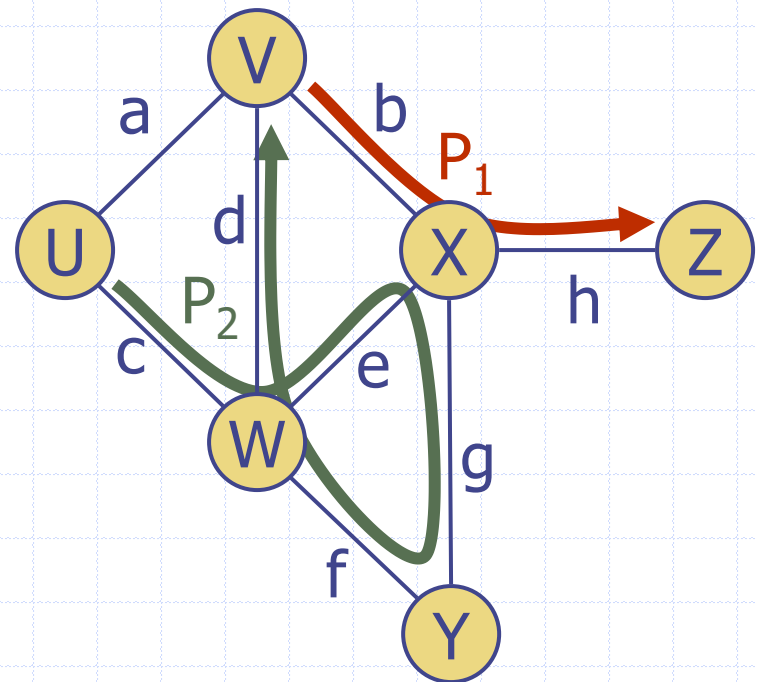
- sequence of alternating vertices and edges
- begins with a vertex
- ends with a vertex
- each edge is preceded and followed by its endpoints

□ Simple path

- all vertices and edges are distinct

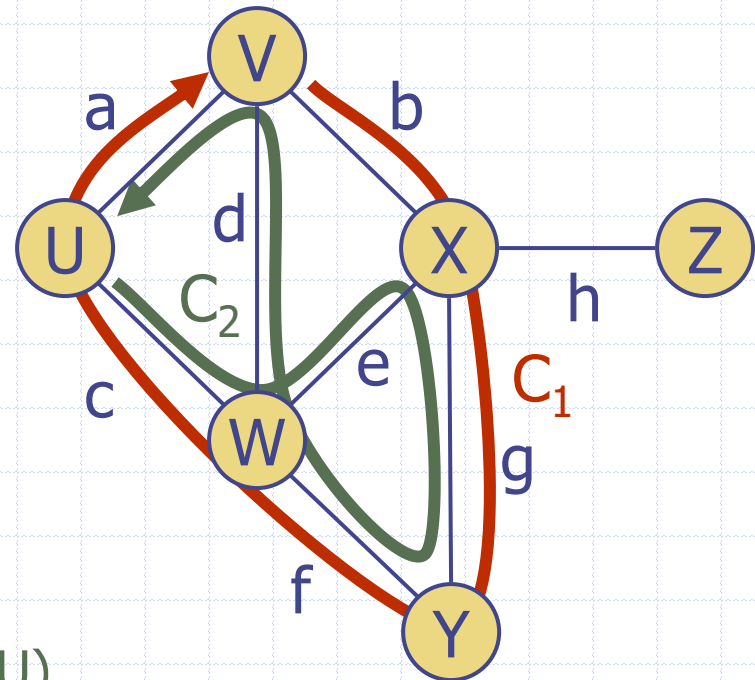
□ Examples

- $P_1 = (V, b, X, h, Z)$ is a simple path
- $P_2 = (U, c, W, e, X, g, Y, f, W, d, V)$ is not a simple path



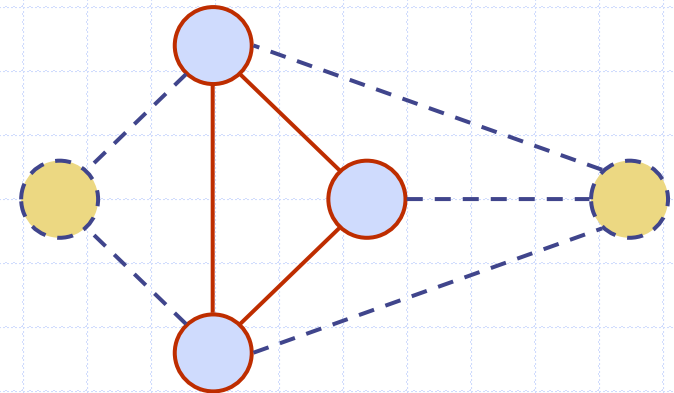
Terminology (cont.)

- **Cycle** cannot move btwn two vertices back N forth only
 - path that starts and ends on the same vertex
- **Simple cycle**
 - all vertices are distinct, except the first and last
 - all edges are distinct
- **Examples**
 - $C_1 = (V, b, X, g, Y, f, W, c, U, a, V)$
 - ◆ a simple cycle
 - $C_2 = (U, c, W, e, X, g, Y, f, W, d, V, a, U)$
 - ◆ not a simple cycle

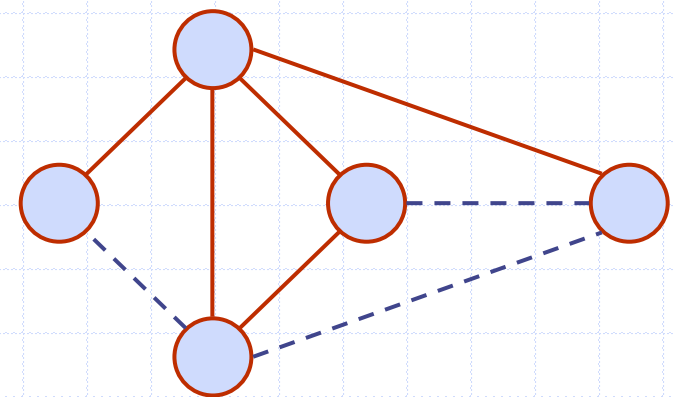


Subgraphs

- Subgraph S of a graph G is a graph such that
 - vertices of S are a subset of the vertices of G
 - edges of S are a subset of the edges of G
- Spanning subgraph of G is a subgraph that contains all the vertices of G



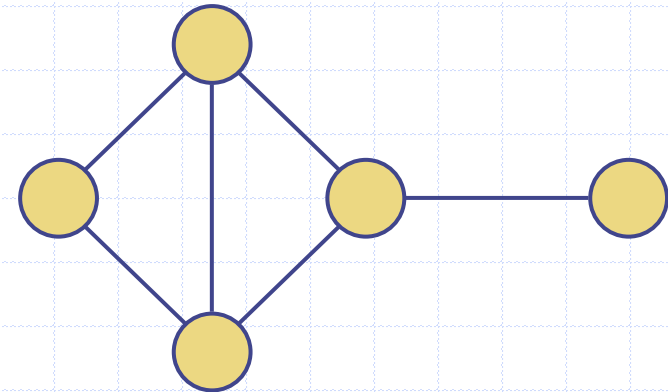
Subgraph



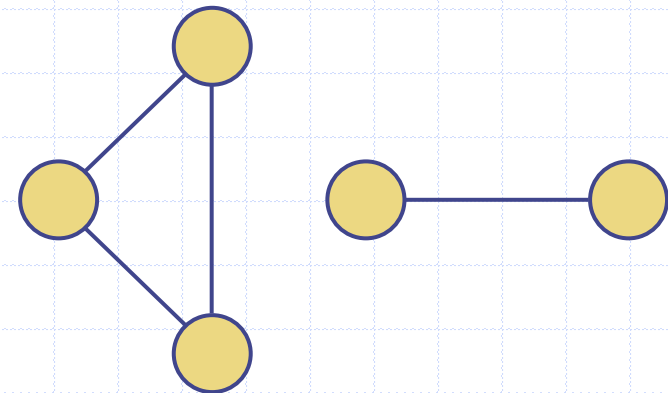
Spanning Subgraph

Connectivity

- Graph is connected if there is a simple path between every pair of vertices
- Connected component of a graph G is a connected subgraph of G



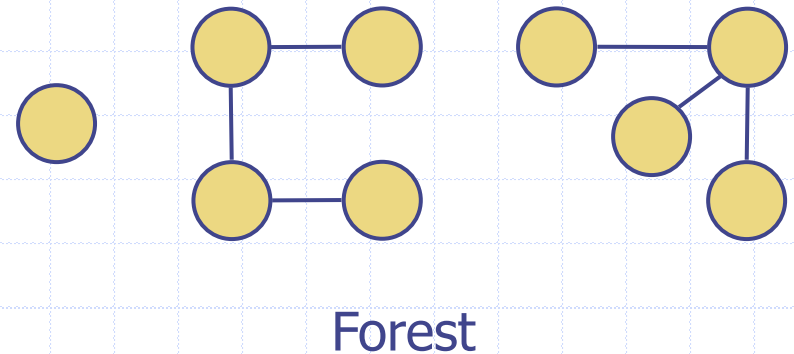
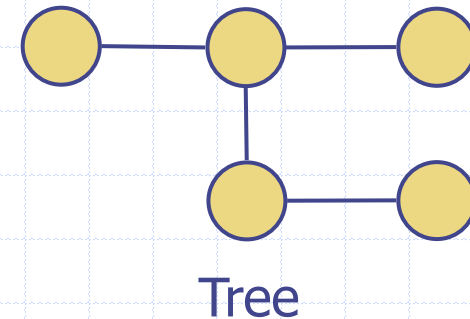
Connected graph



Non connected graph with two connected components

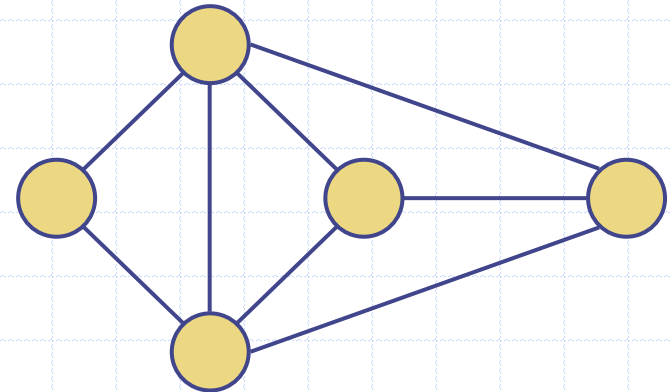
Trees and Forests

- Undirected graph T is an unrooted tree if
 - T is connected
 - T has no simple cycles
- An undirected graph is a forest if it contains no simple cycles

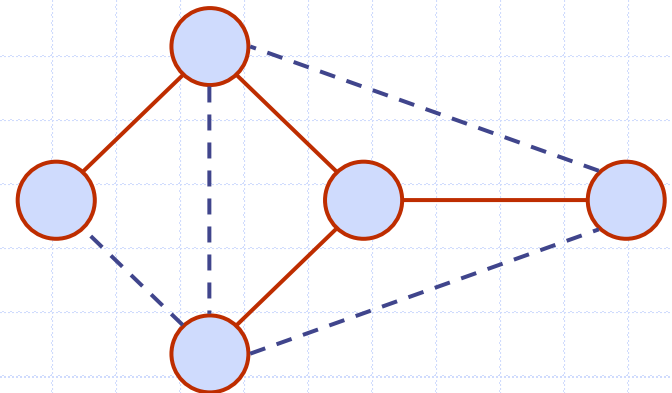


Spanning Trees and Forests

- Spanning tree of a connected graph is a spanning subgraph that is a tree
- Spanning forest of a graph is a spanning subgraph that is a forest



Graph



Spanning tree

Properties

$$\sum_v \deg(v) = 2m$$

- Proof: each edge is counted twice

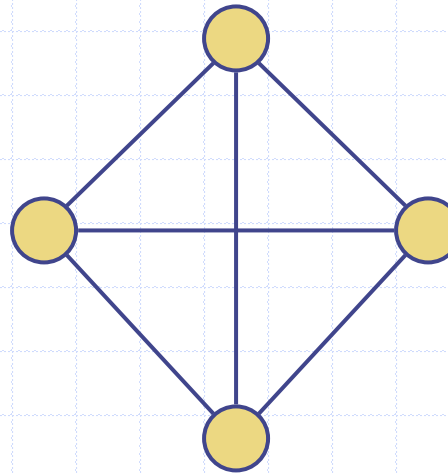
In an undirected graph with no self-loops and no multiple edges
 $m \leq n(n-1)/2$

Notation

n	number of vertices
m	number of edges
$\deg(v)$	degree of vertex v

Example

- $n = 4$
- $m = 6$
- $\deg(v) = 3$



Graph Density

- Undirected simple graph

- $D = \frac{2m}{n(n-1)}$

- Directed simple graph

- $D = \frac{m}{n(n-1)}$

- Maximum density is 1

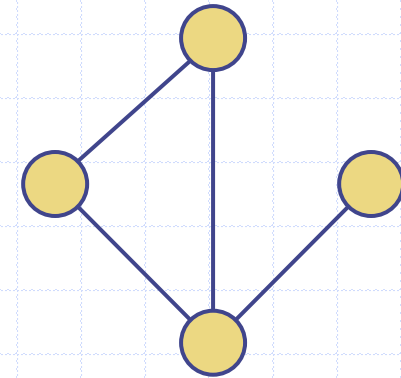
- Minimum density is 0

Graph Density

□ Sparse

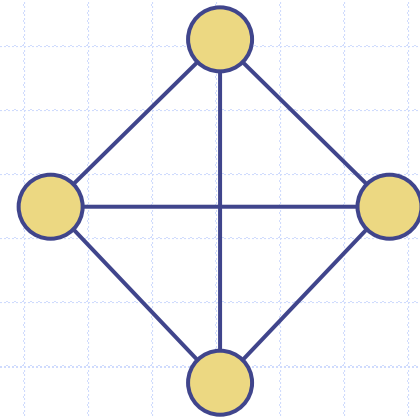
- $m \cong O(n)$ order of vertices

on avg: every vertex has one out going edge



□ Dense

- $m \cong O(n^2)$



Week 8 – Graphs

1. Graph fundamentals
2. Graph implementation
3. Depth-first search
4. Breadth-first search
5. Directed graphs

Vertex and Edge ADTs

- **Graph** is a collection of **vertices** and **edges**
- Model the abstraction as a combination of three data types: Vertex, Edge and Graph
- **Vertex** is a lightweight object that stores an arbitrary element provided by the user (e.g. an airport code)
 - **element()** method retrieves stored element
- **Edge** stores an associated object (e.g. a flight number, travel distance, cost)
 - retrieved with **element()**

Graph ADT

- `numVertices()`: Returns the number of vertices of the graph.
- `vertices()`: Returns an iteration of all the vertices of the graph.
- `numEdges()`: Returns the number of edges of the graph.
- `edges()`: Returns an iteration of all the edges of the graph.
- `getEdge( $u, v$ )`: Returns the edge from vertex u to vertex v , if one exists; otherwise return null. For an undirected graph, there is no difference between `getEdge( $u, v$ )` and `getEdge( $v, u$ )`.
- `endVertices( $e$ )`: Returns an array containing the two endpoint vertices of edge e . If the graph is directed, the first vertex is the origin and the second is the destination.
- `opposite( $v, e$ )`: For edge e incident to vertex v , returns the other vertex of the edge; an error occurs if e is not incident to v .
- `outDegree( $v$ )`: Returns the number of outgoing edges from vertex v .
- `inDegree( $v$ )`: Returns the number of incoming edges to vertex v . For an undirected graph, this returns the same value as `outDegree( $v$ )`.

Graph ADT

- `outgoingEdges( $v$ )`: Returns an iteration of all outgoing edges from vertex v .
- `incomingEdges( $v$ )`: Returns an iteration of all incoming edges to vertex v . For an undirected graph, this returns the same collection as does `outgoingEdges( $v$ )`.
- `insertVertex( $x$ )`: Creates and returns a new Vertex storing element x .
- `insertEdge( $u, v, x$ )`: Creates and returns a new Edge from vertex u to vertex v , storing element x ; an error occurs if there already exists an edge from u to v .
- `removeVertex( $v$ )`: Removes vertex v and all its incident edges from the graph.
- `removeEdge( $e$ )`: Removes edge e from the graph.

Edge List

- Sequence of edges
 - represented by vertex pairs

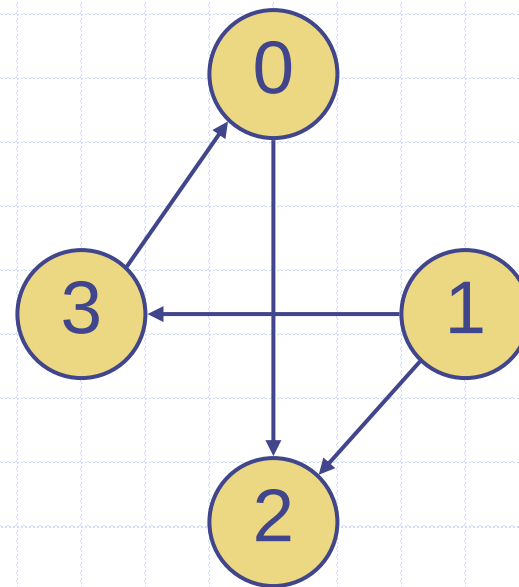
Edge List

(1,2)

(1,3)

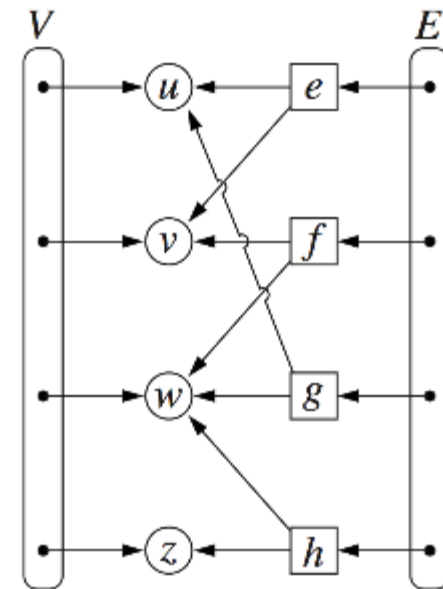
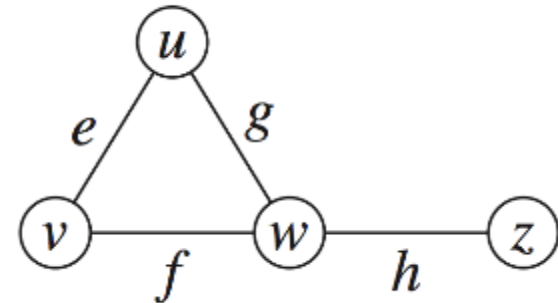
(3,0)

(0,2)



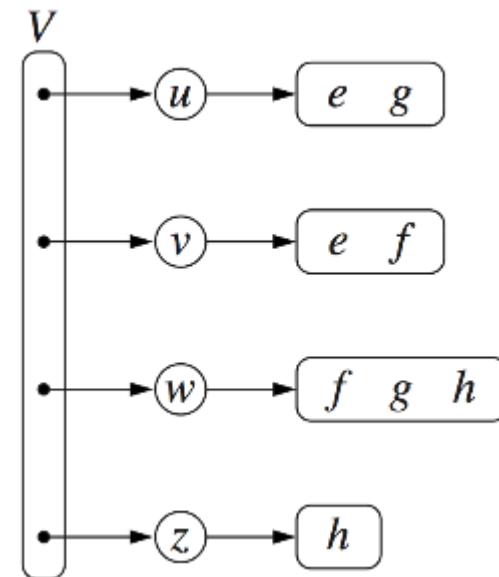
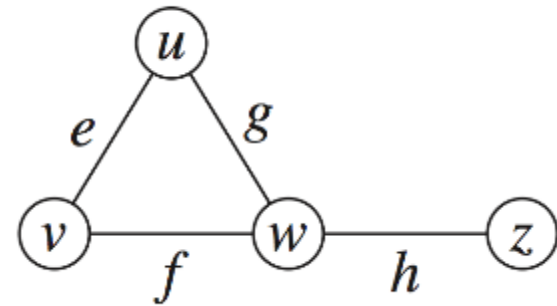
Edge List Structure

- Vertex object
 - element
 - reference to position in vertex sequence
- Edge object
 - element
 - origin vertex object
 - destination vertex object
 - reference to position in edge sequence
- Vertex sequence
 - sequence of vertex objects
- Edge sequence
 - sequence of edge objects



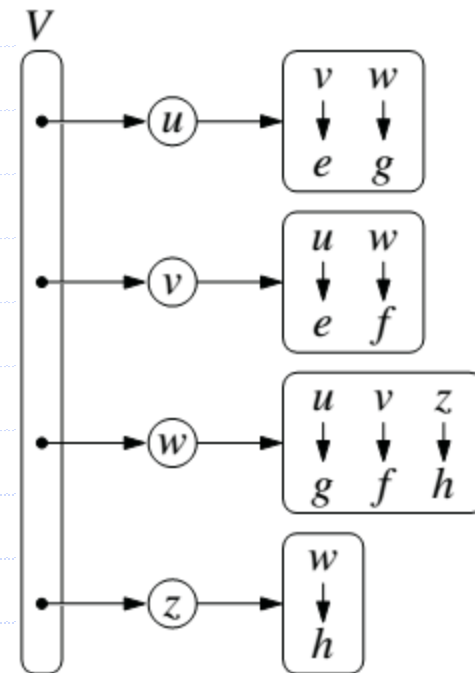
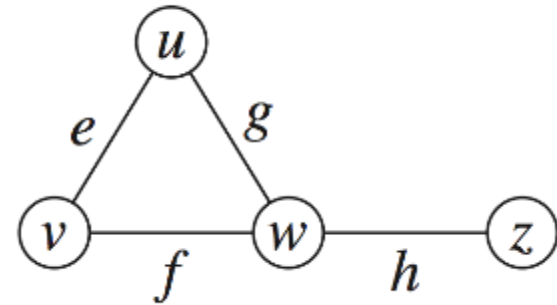
Adjacency List Structure

- Extends edge list
- Incidence sequence for each vertex
 - sequence of references to edge objects of incident edges
- Augmented edge objects
 - references to associated positions in incidence sequences of end vertices



Adjacency Map Structure

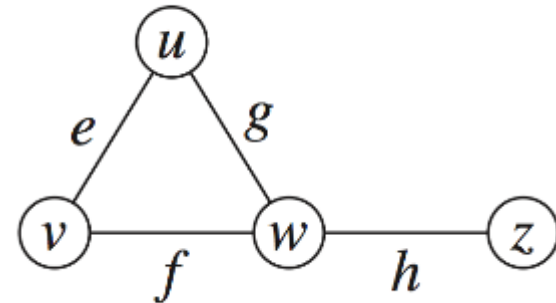
- Adapts adjacency list
- Incidence list becomes a map



Adjacency Matrix Structure

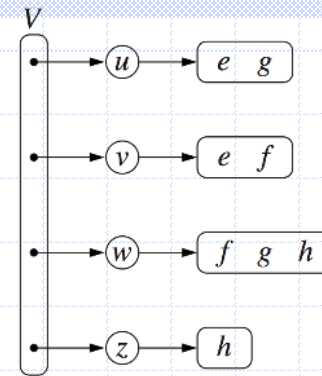
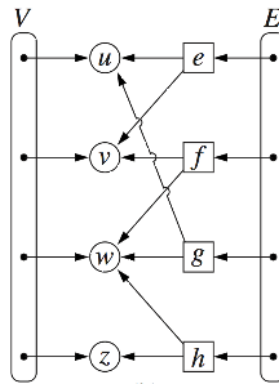
useful if the graph is dense

- Edge list structure
- Augmented vertex objects
 - integer key (index) associated with vertex
- 2D-array adjacency matrix
 - reference to edge object for adjacent vertices
 - null for nonadjacent vertices
- “Old fashioned” version
 - 0 for no edge
 - 1 for edge



		0	1	2	3
u	→ 0		e	g	
v	→ 1	e		f	
w	→ 2	g	f		h
z	→ 3			h	

Performance



		0	1	2	3
$u \rightarrow$	0		e	g	
$v \rightarrow$	1	e		f	
$w \rightarrow$	2	g	f		h
$z \rightarrow$	3			h	

▪ n vertices, m edges	Edge List	Adjacency List	Adjacency Matrix
Space	$n + m$	$n + m$	n^2
outgoingEdges(v) incomingEdges(v)	m	deg(v)	n
getEdge(v, w)	m	min(deg(v), deg(w))	1
insertVertex(o)	1	1	n^2
insertEdge(v, w, o)	1	1	1
removeVertex(v)	m	deg(v)	n^2
removeEdge(e)	1	1	1

Week 8 – Graphs

1. Graph fundamentals
2. Graph implementation
3. Depth-first search
4. Breadth-first search
5. Directed graphs

Graph Traversal

- Systematic process for exploring a graph
 - visit all vertices and edges
- Efficient if done in linear time
- Allows determination of **reachability**
 - how to travel from one vertex to another

Depth-First Search (DFS)

- General technique for traversing a graph
- DFS traversal of a graph G : $O(n + m)$
 - visits all the vertices and edges of G
 - determines whether G is connected
 - computes the connected components of G
 - computes a spanning forest of G

DFS Algorithm from a Vertex

Algorithm *DFS*(G, v):

Input Graph G and a vertex v of G

Output Collection of vertices reachable from v and their discovery edges and back edges

Mark vertex v as visited

for all $e \in G.outgoingEdges(v)$ **do**

if e is not explored **then**

$w \leftarrow G.opposite(v, e)$

if w has not been visited **then**

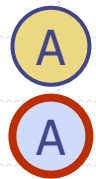
 Record edge e as a discovery edge for vertex w

DFS(G, w)

else

 Mark e as a back edge for vertex w

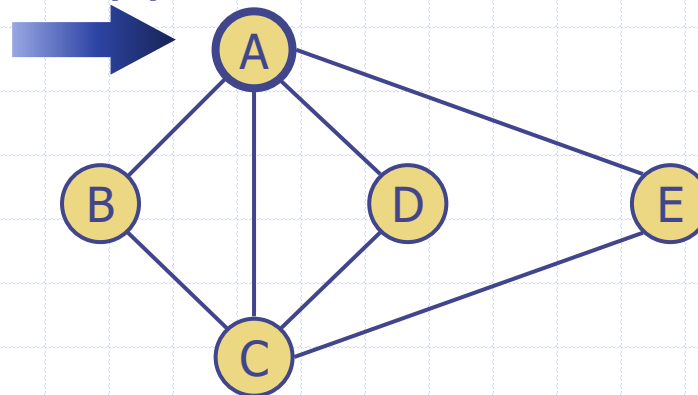
Example



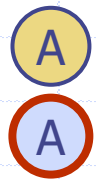
unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

DFS(A)



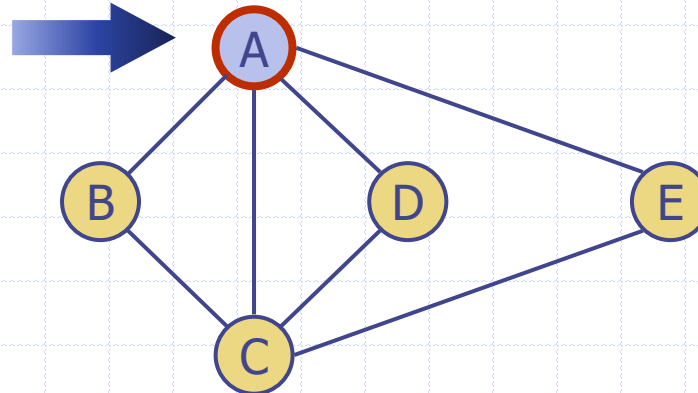
Example



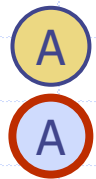
unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

DFS(A)

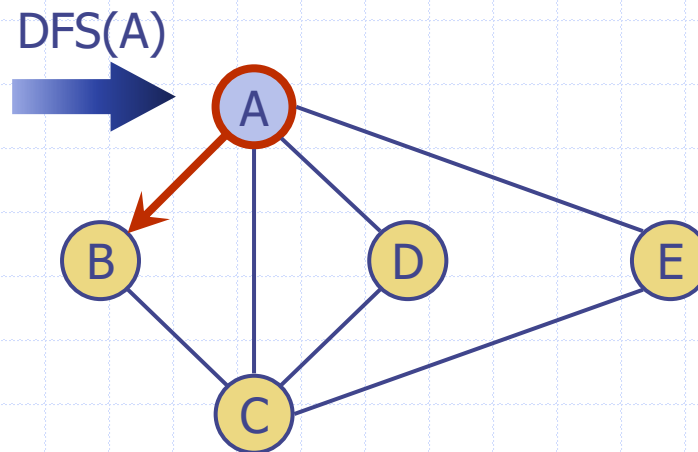


Example

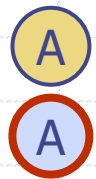


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

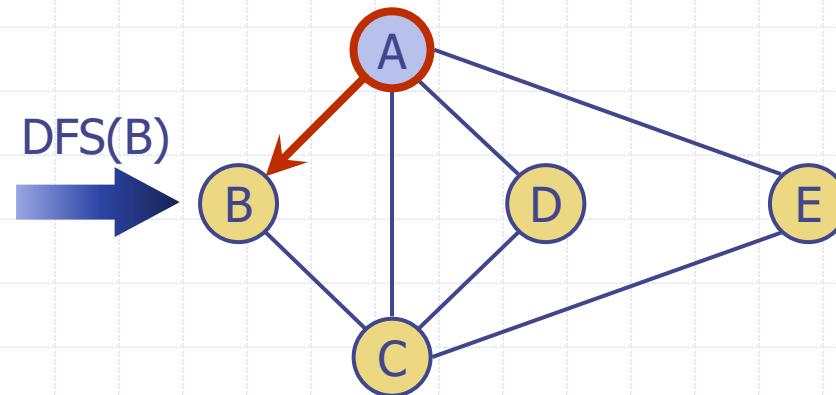


Example

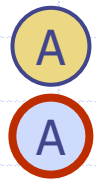


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

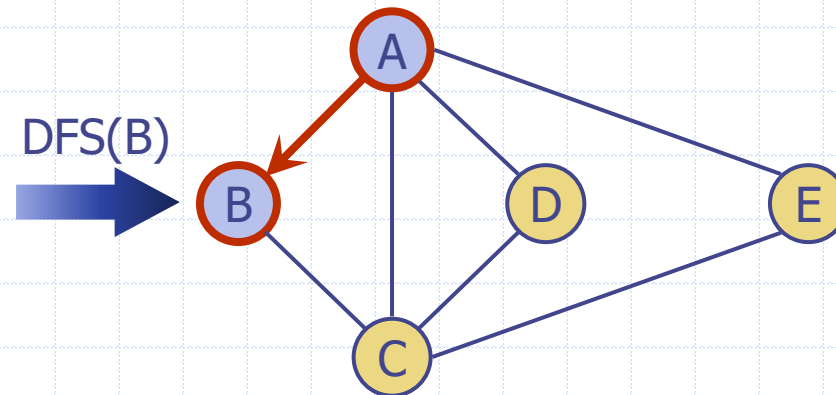


Example

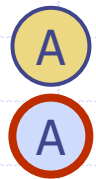


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

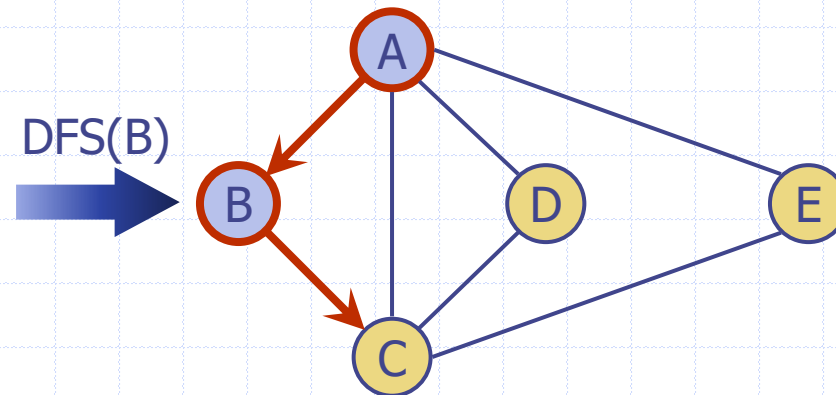


Example

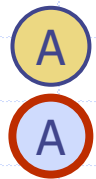


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

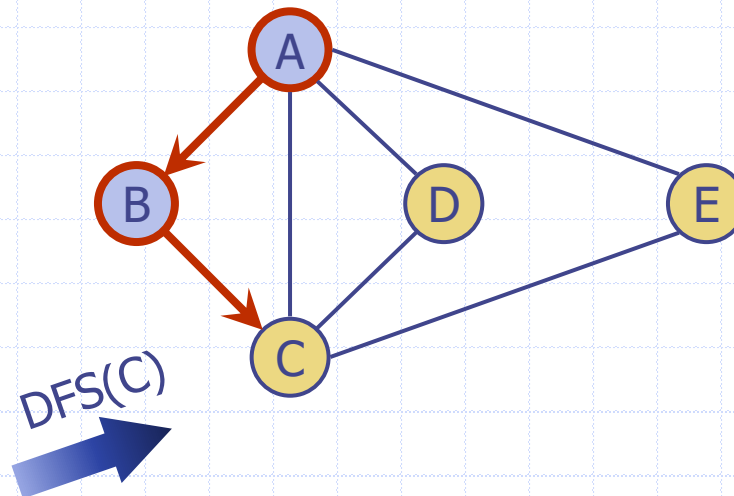


Example

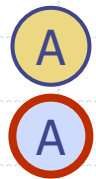


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

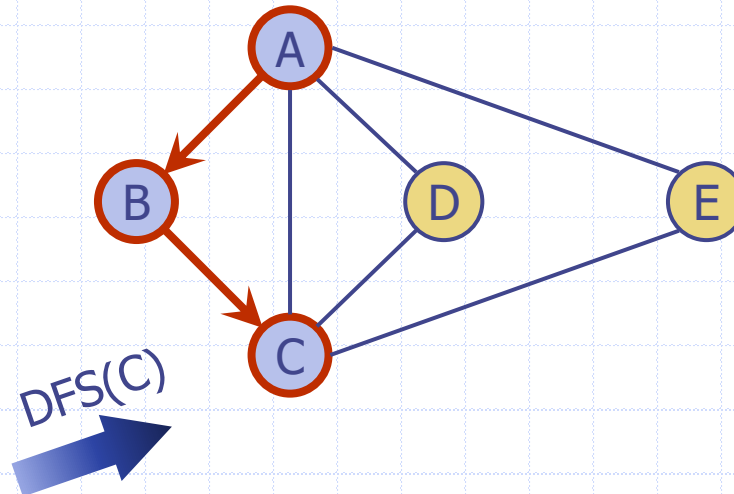


Example

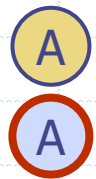


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

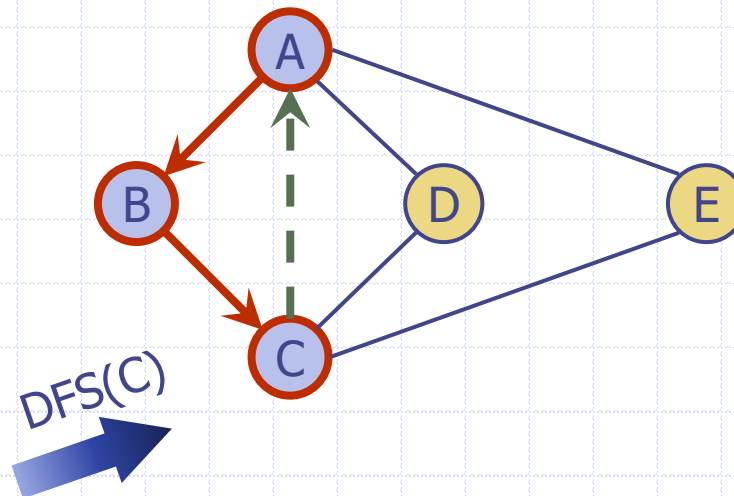


Example

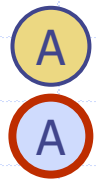


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

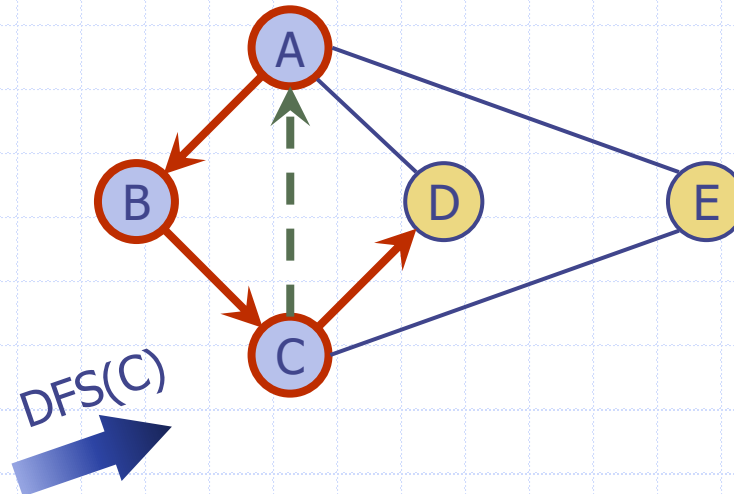


Example

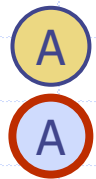


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge



Example



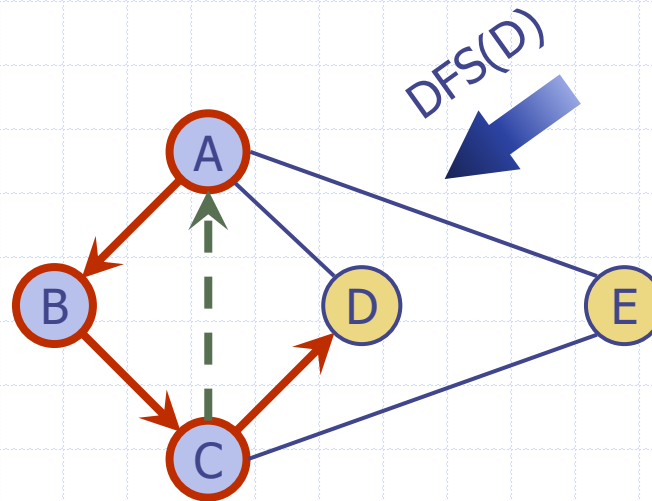
unexplored vertex

visited vertex

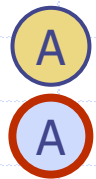
unexplored edge

discovery edge

back edge

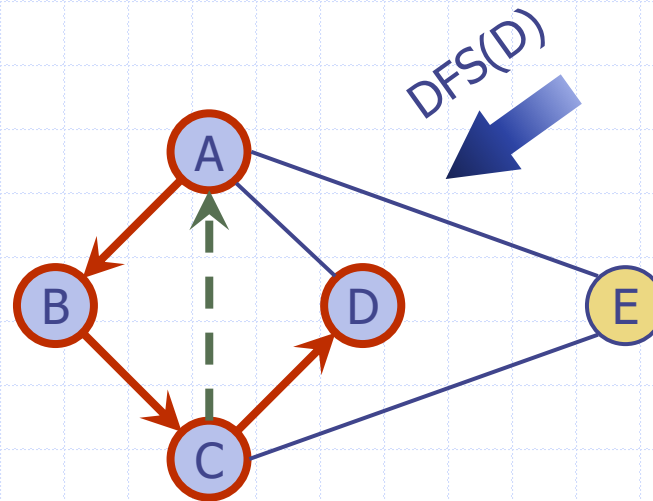


Example

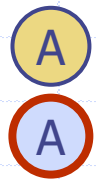


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

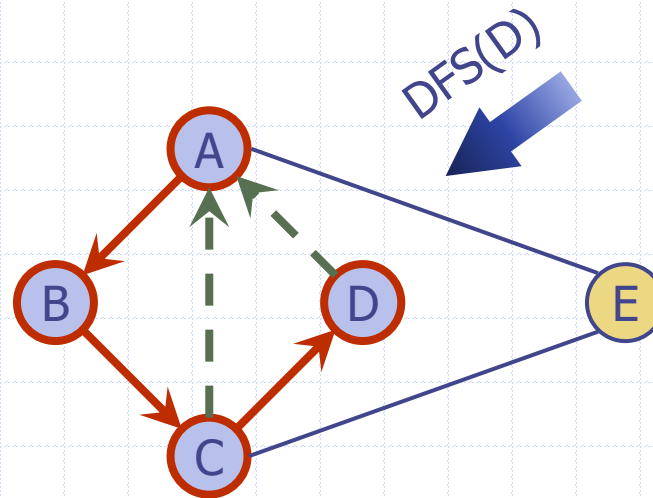


Example

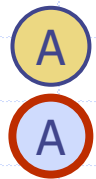


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

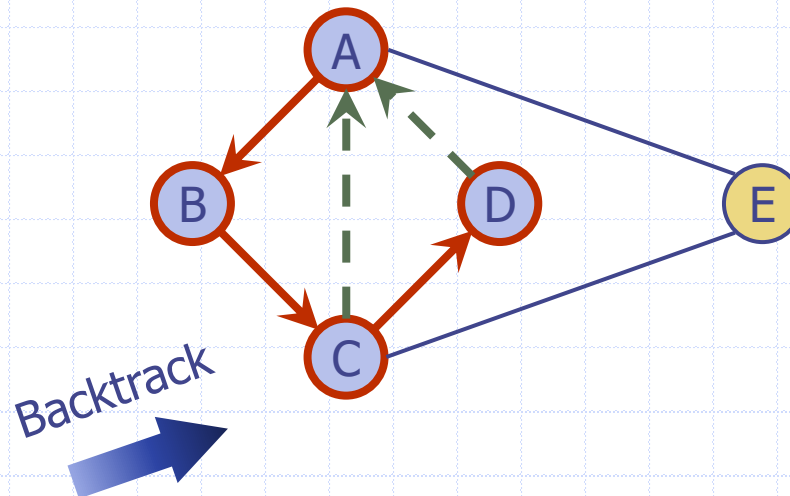


Example

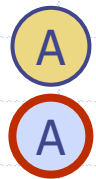


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

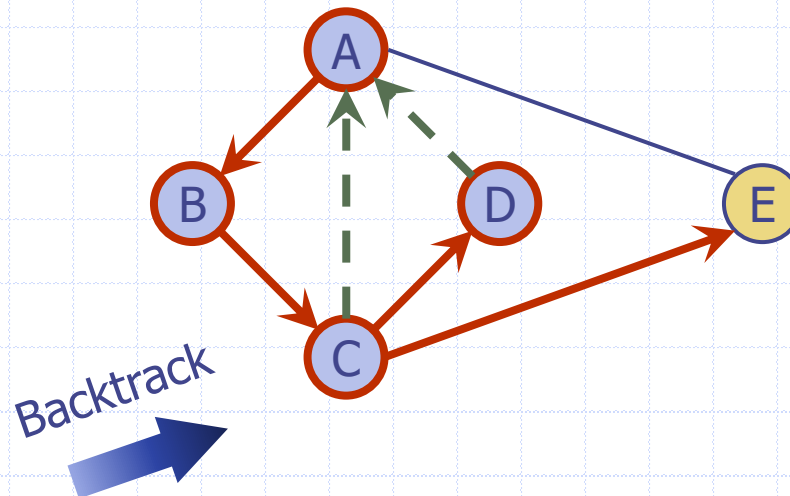


Example

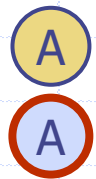


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

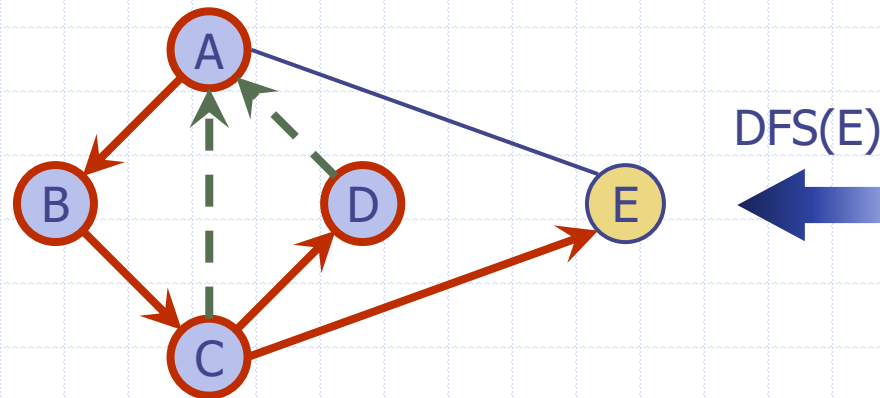


Example

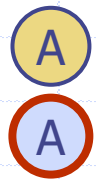


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

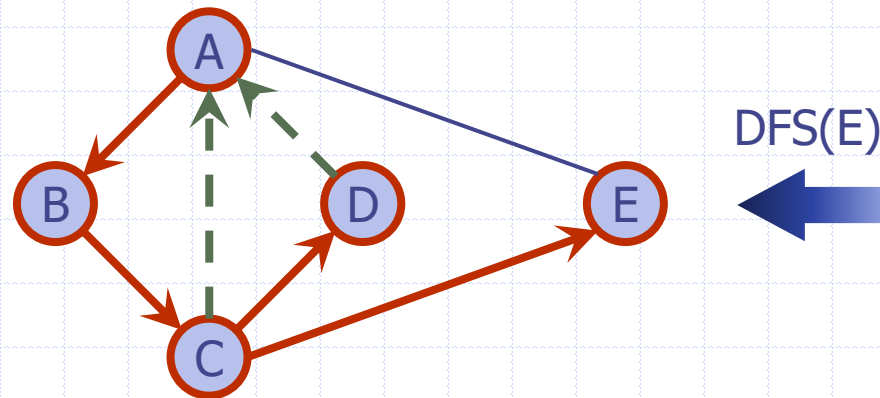


Example

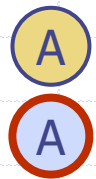


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge



Example



unexplored vertex

visited vertex

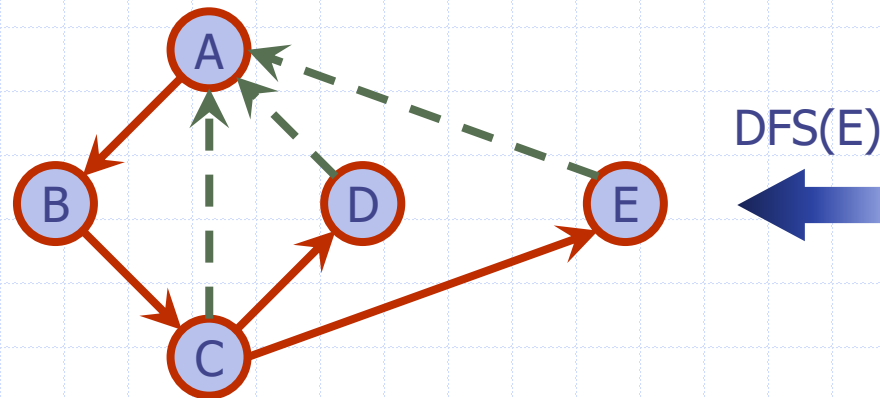


unexplored edge

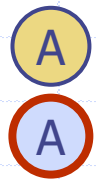
discovery edge



back edge

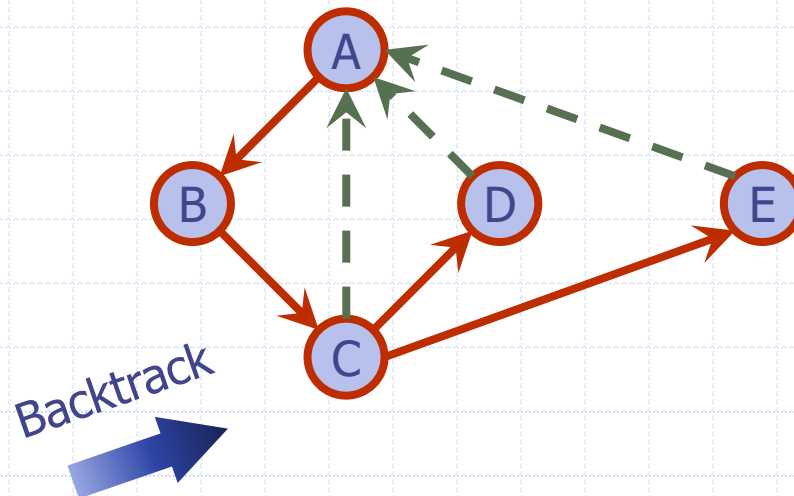


Example

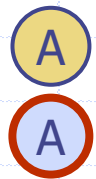


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

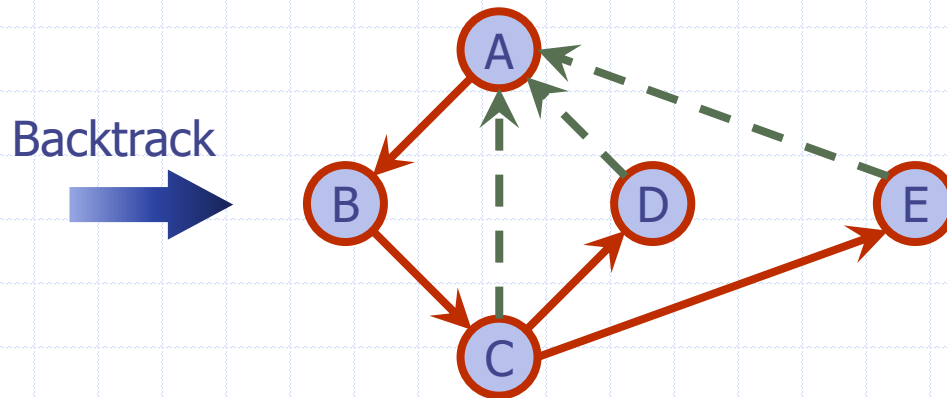


Example

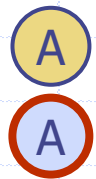


unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge



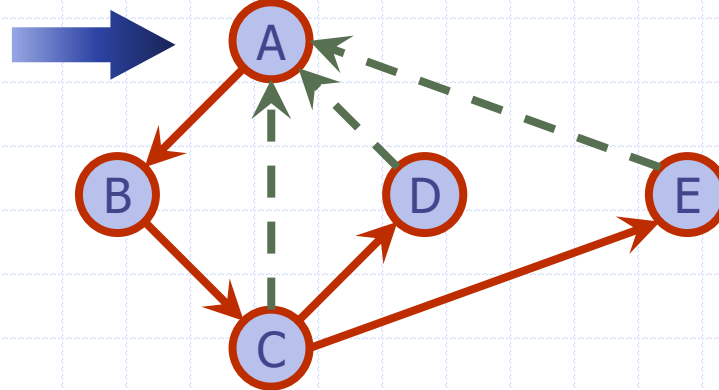
Example



unexplored vertex
visited vertex

unexplored edge
discovery edge
back edge

Backtrack



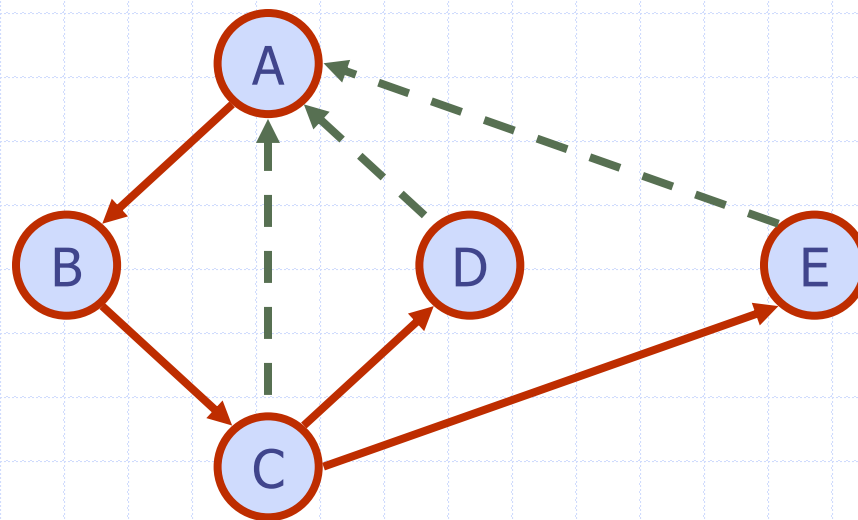
Properties of DFS

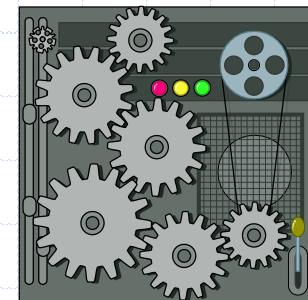
Property 1

$DFS(G, v)$ visits all the vertices and edges in the connected component of v

Property 2

Discovery edges labelled by $DFS(G, v)$ form a spanning tree of the connected component of v





Analysis of DFS

- ❑ Setting/getting a vertex/edge label takes $O(1)$ time
- ❑ Each vertex is labelled twice
 - once as UNEXPLORED
 - once as **VISITED**
- ❑ Each edge is labelled twice
 - once as UNEXPLORED
 - once as **DISCOVERY** or **BACK**
- ❑ Method outgoingEdges is called once for each vertex
- ❑ DFS runs in $O(n + m)$ time
 - provided graph is implemented as an adjacency list
 - ◆ recall that $\sum_v \deg(v) = 2m$

Path Finding

Algorithm *pathDFS*(G, v, S, z):

Mark vertex v as visited

S.addLast(v)

if $v = z$ **then return** **true**

for all $e \in G.outgoingEdges(v)$ **do**

if e is unexplored **then**

$w \leftarrow G.opposite(v, e)$

if w has not been visited **then**

 Record edge e as a discovery edge for vertex w

S.addLast(e)

if *pathDFS*(G, w, S, z) **then**

return **true**

S.removeLast()

S.removeLast()

return **false**

Cycle Finding

Algorithm *cycleDFS*(G, v, S):

Mark vertex v as visited

S.addLast(v)

for all $e \in G.outgoingEdges(v)$ **do**

if e is unexplored **then**

$w \leftarrow G.opposite(v, e)$

if w has not been visited **then**

 Record edge e as a discovery edge for vertex w

S.addLast(e)

if *cycleDFS*(G, w, S) **then**

return true

S.removeLast()

else

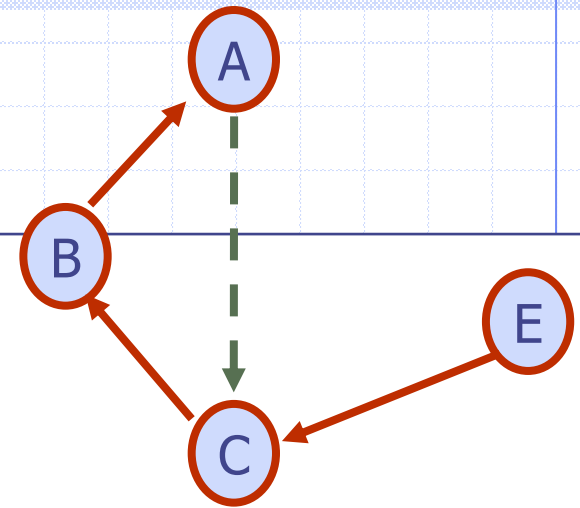
 Remove prefix of S up to first occurrence of w

S.addLast(w)

return true

S.removeLast()

return false



DFS for Entire Graph

Algorithm *DFS*(*G*):

Input Graph *G*

Output Labelling of edges of *G* as discovery and back edges

for all *u* $\in$ *G.vertices()* **do**

 Set *u* to be unvisited

for all *e* $\in$ *G.edges()* **do**

 Set *e* to be unexplored

for all *v* $\in$ *G.vertices()* **do**

if *v* has not been visited **then**

DFS(*G*, *v*)

Week 8 – Graphs

1. Graph fundamentals
2. Graph implementation
3. Depth-first search
4. Breadth-first search
5. Directed graphs

Breadth-First Search

- General technique for traversing a graph
- BFS traversal of a graph G
 - visits all vertices and edges of G
 - determines whether G is connected
 - computes the connected components of G
 - computes a spanning forest of G

BFS Algorithm from a Vertex

Algorithm *BFS*(G, u)

Input Graph G and a vertex u of G

Output Collection of vertices reachable from u and their discovery and cross edges

Q = new empty queue

$Q.enqueue(u)$

Mark vertex u as visited

while $\neg Q.isEmpty()$ **do**

$v = Q.dequeue()$

for all $e \in G.incidentEdges(v)$ **do**

if e is not explored **then**

$w \leftarrow G.opposite(v, e)$

if w has not been visited **then**

 Record edge e as a discovery edge for vertex w

$Q.enqueue(w)$

 Mark vertex w as visited

else

 Mark e as a cross edge

Visits vertices in order of their distance (in edges) from the start vertex!

BFS for Entire Graph

Algorithm *BFS*(*G*):

Input Graph *G*

Output Labelling of edges of *G* as discovery and cross edges

for all *u* $\in$ *G.vertices()* **do**

 Set *u* to be unvisited

for all *e* $\in$ *G.edges()* **do**

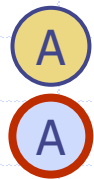
 Set *e* to be unexplored

for all *v* $\in$ *G.vertices()* **do**

if *v* has not been visited **then**

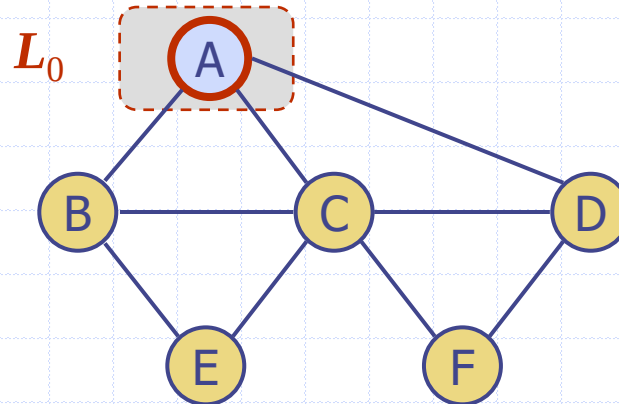
BFS(*G*, *v*)

Example



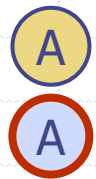
unexplored vertex
visited vertex

unexplored edge
discovery edge
cross edge



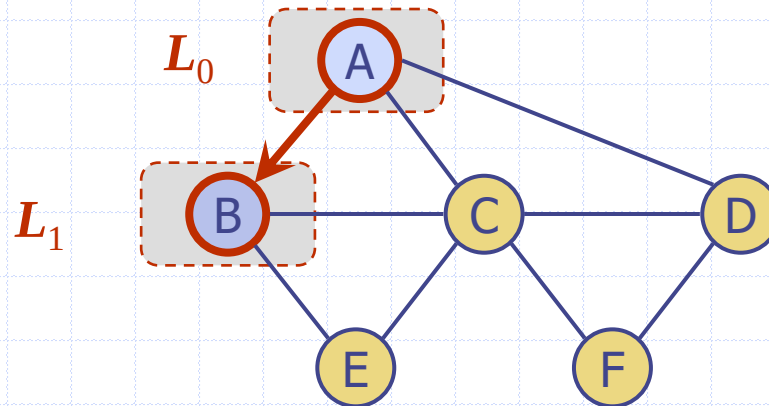
$Q = [A]$

Example



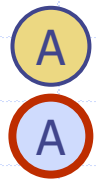
unexplored vertex
visited vertex

— unexplored edge
→ discovery edge
- - - cross edge

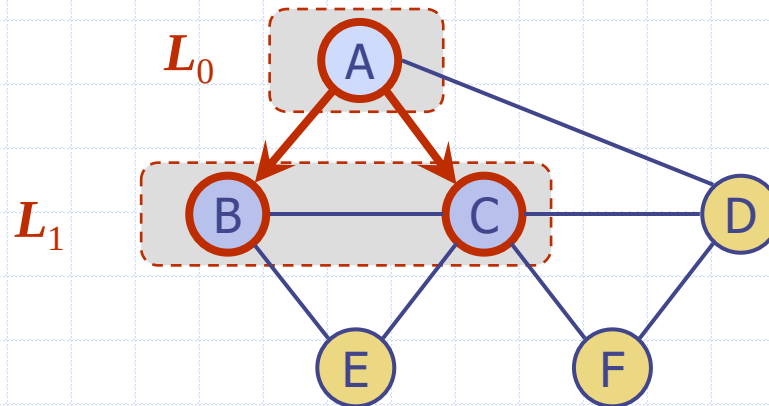
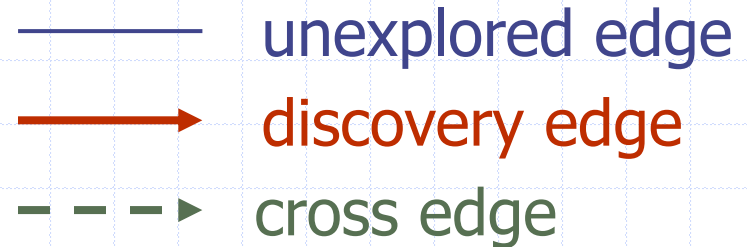


$Q = [A, B]$

Example

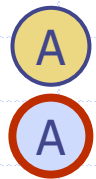


unexplored vertex
visited vertex

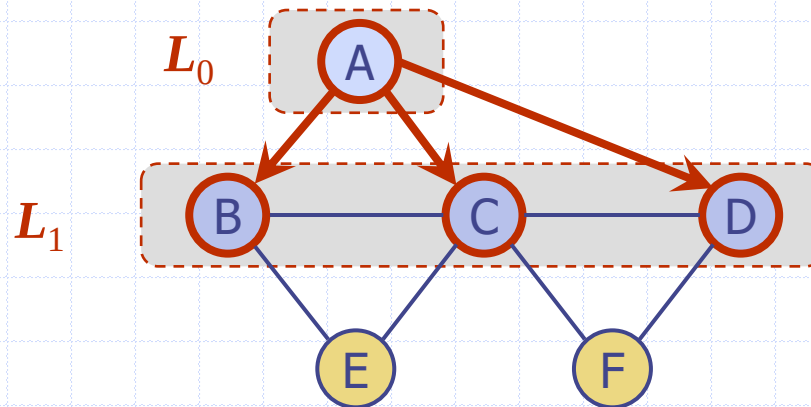
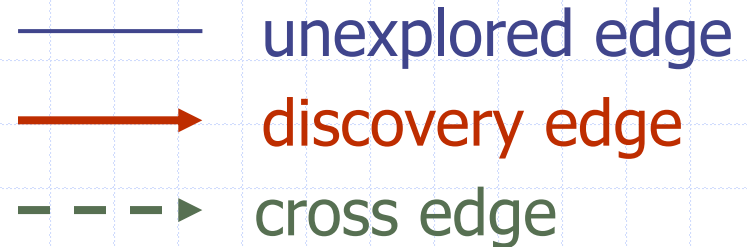


$Q = [A, B, C]$

Example

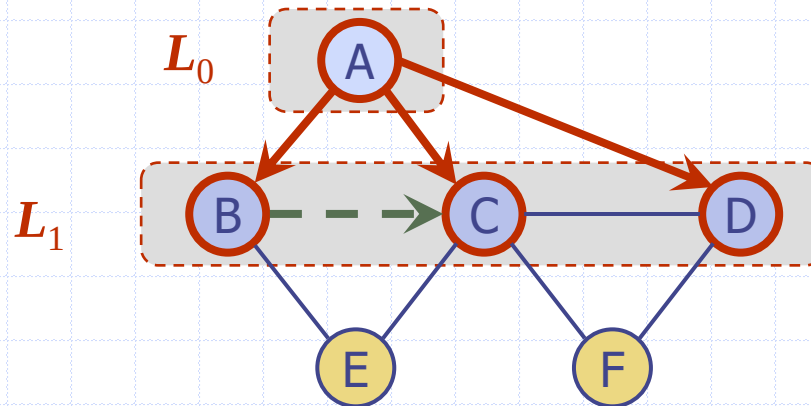
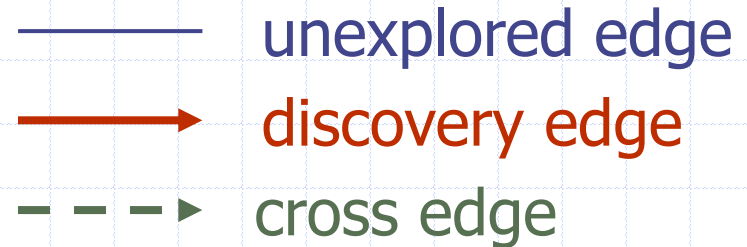
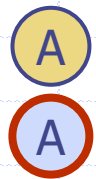


unexplored vertex
visited vertex



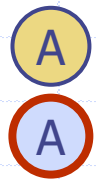
$Q = [A, B, C, D]$

Example



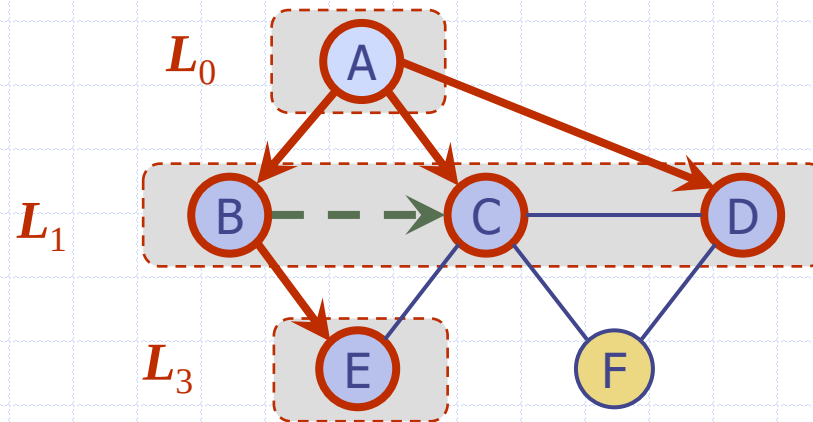
$Q = [\text{A}, \text{B}, \text{C}, \text{D}]$

Example



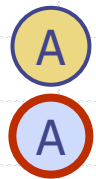
unexplored vertex
visited vertex

unexplored edge
discovery edge
cross edge



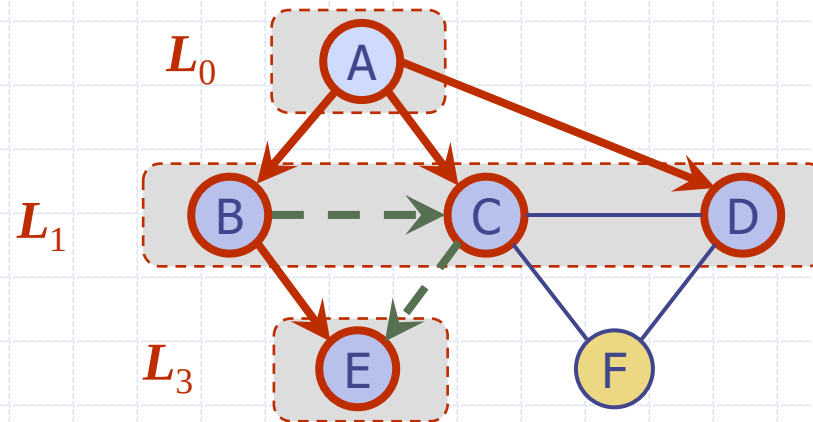
$Q = [\text{A}, \text{B}, \text{C}, \text{D}, \text{E}]$

Example



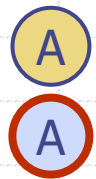
unexplored vertex
visited vertex

unexplored edge
discovery edge
cross edge

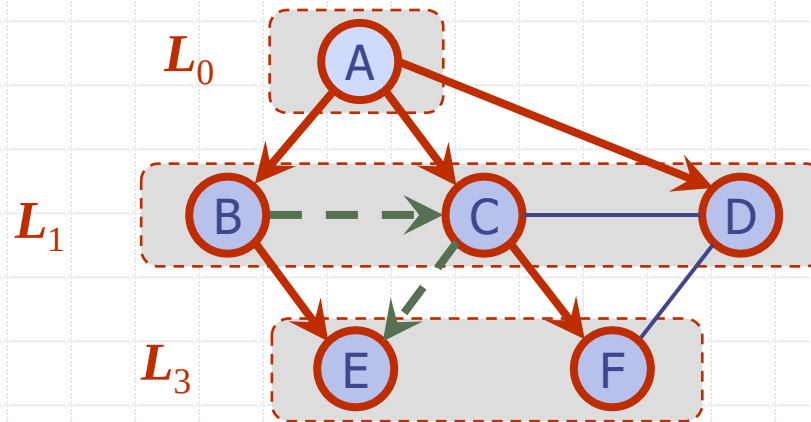


$Q = [\text{A, B, C, D, E}]$

Example

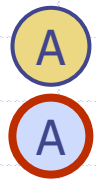


unexplored edge
discovery edge
cross edge



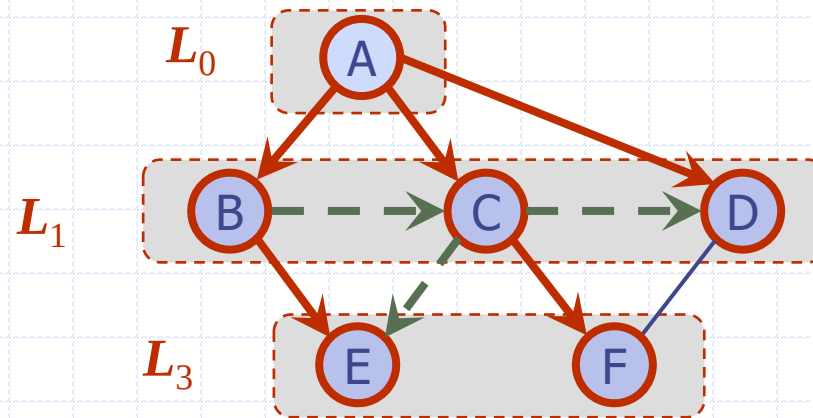
$Q = [\text{A, B, C, D, E, F}]$

Example



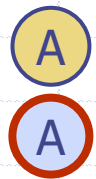
unexplored vertex
visited vertex

unexplored edge
discovery edge
cross edge

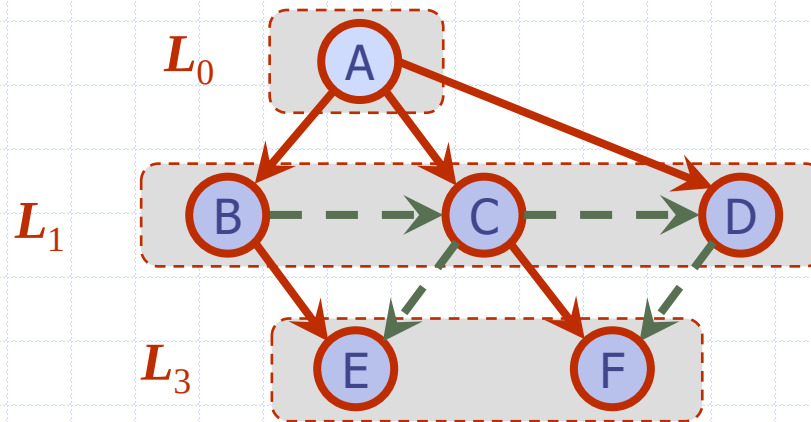


$Q = [\text{A, B, C, D, E, F}]$

Example

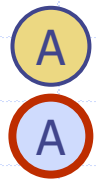


unexplored edge
discovery edge
cross edge

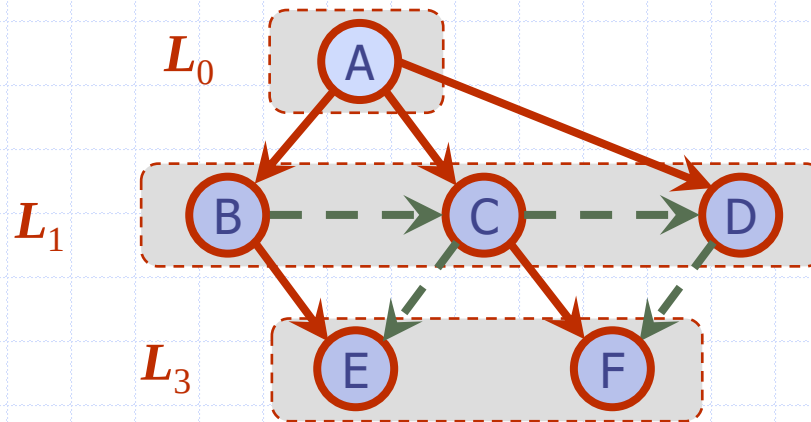


$Q = [\text{A}, \text{B}, \text{C}, \text{D}, \text{E}, \text{F}]$

Example

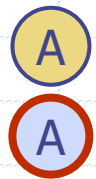


unexplored edge
discovery edge
cross edge



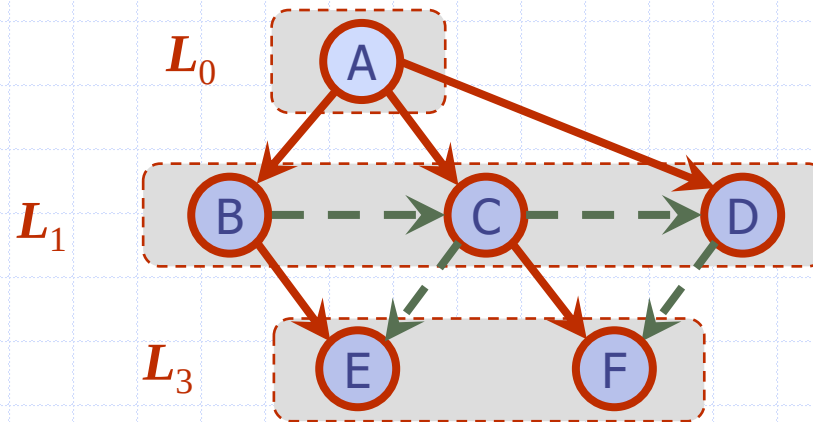
$Q = [A, B, C, D, E, F]$

Example



unexplored vertex
visited vertex

unexplored edge
discovery edge
cross edge

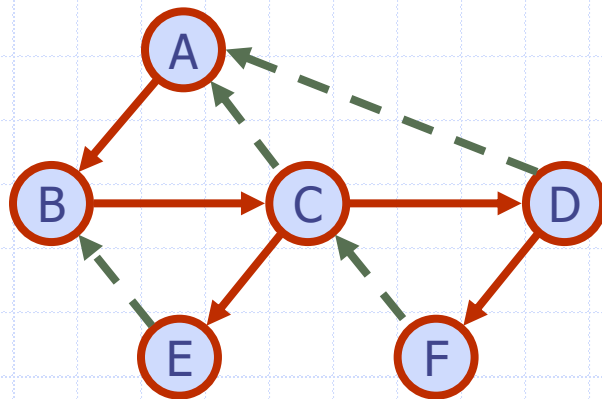


$Q = [\text{A, B, C, D, E, F}]$

DFS vs. BFS

Back edge (v, w)

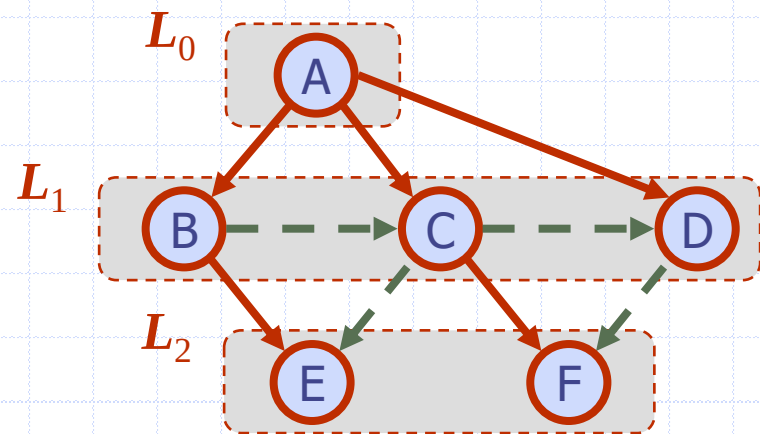
- w is an ancestor of v in the tree of discovery edges



DFS

Cross edge (v, w)

- w is in the same level as v , or in the next level



BFS

Properties

Notation

G_s – connected component of s

Property 1

$BFS(G, s)$ visits all the vertices and edges of G_s

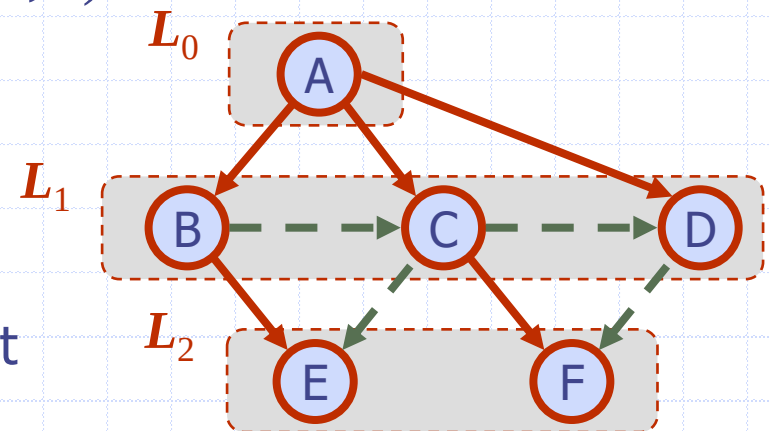
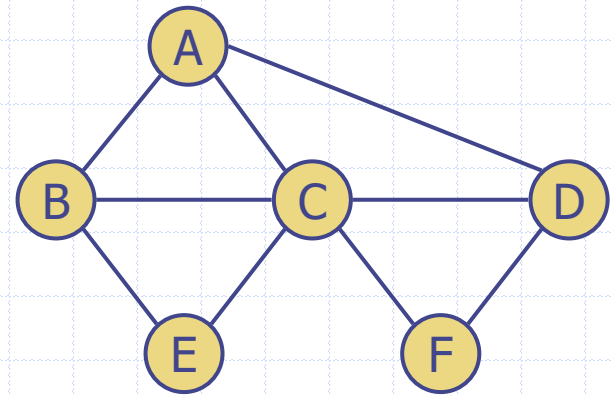
Property 2

Discovery edges labeled by $BFS(G, s)$ form a spanning tree T_s of G_s

Property 3

For each vertex v in L_i

- path of T_s from s to v has i edges
- every path from s to v in G_s has at least i edges



Analysis of BFS

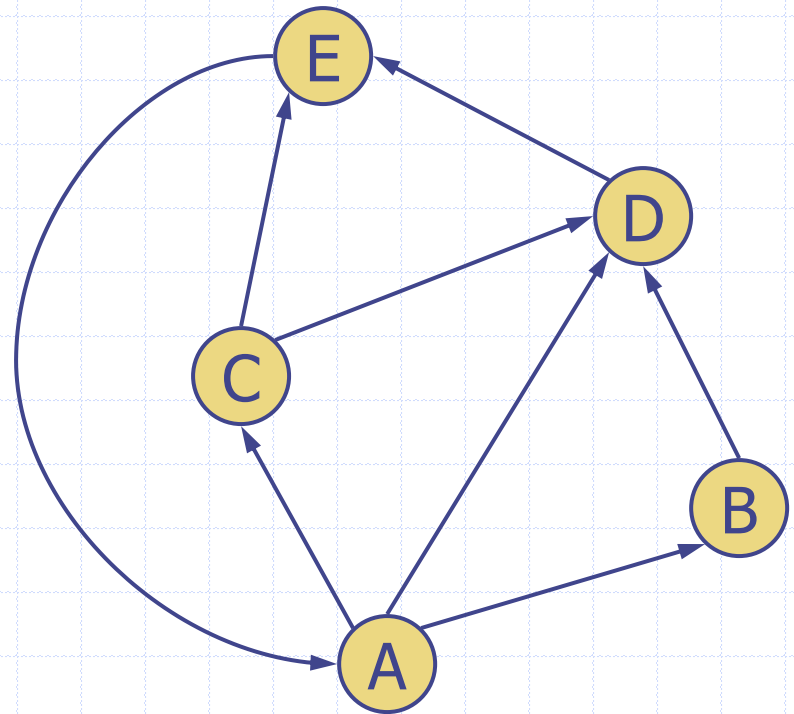
- ❑ Setting/getting a vertex/edge label takes $O(1)$ time
- ❑ Each vertex is labelled twice
 - once as UNEXPLORED
 - once as VISITED
- ❑ Each edge is labelled twice
 - once as UNEXPLORED
 - once as DISCOVERY or CROSS
- ❑ Each vertex is inserted once into a sequence L_i
- ❑ Method outgoingEdges is called once for each vertex
- ❑ BFS runs in $O(n + m)$ time
 - provided graph is implemented as an adjacency list
 - ◆ recall that $\sum_v \deg(v) = 2m$

Week 8 – Graphs

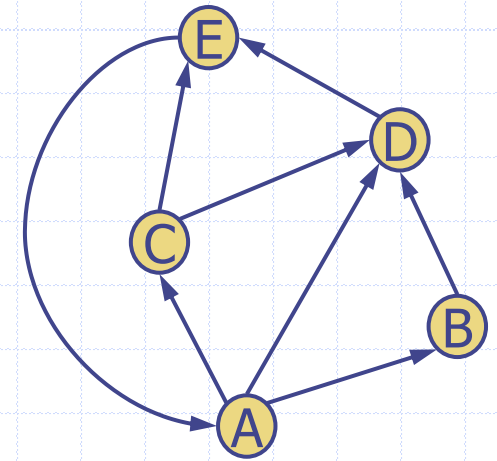
1. Graph fundamentals
2. Graph implementation
3. Depth-first search
4. Breadth-first search
5. Directed graphs

Digraphs

- Graph whose edges are all directed
 - “directed graph”
- Applications
 - one-way streets
 - flights
 - task scheduling



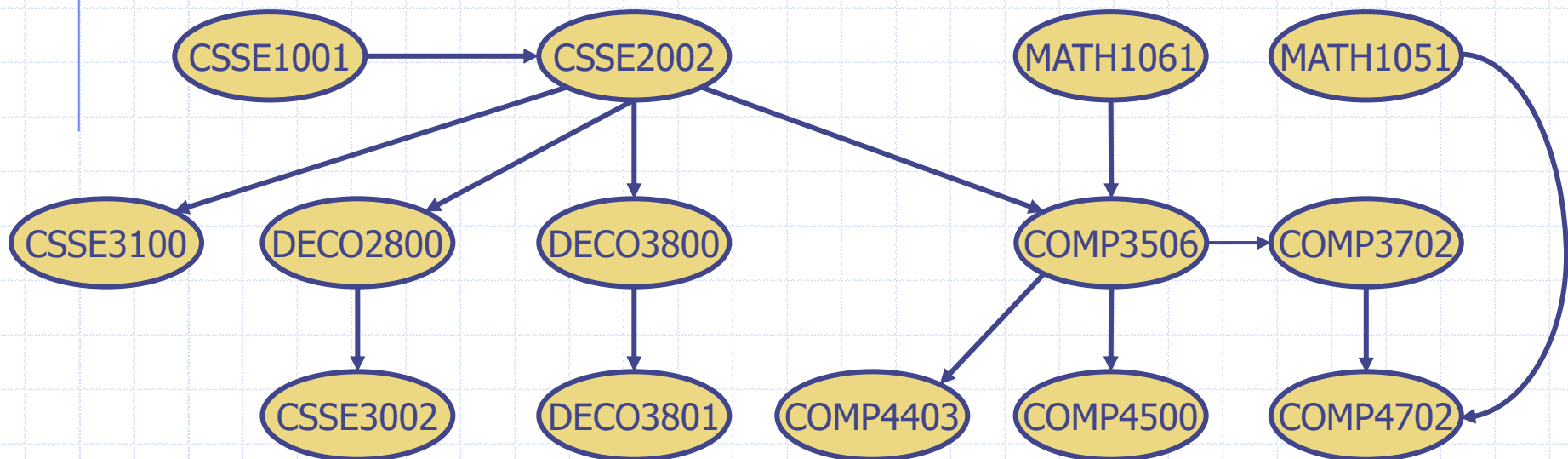
Digraph Properties



- Graph $G=(V,E)$ such that
 - each edge goes in **one direction**
 - edge (a,b) goes from a to b , but **not** b to a
- If G is simple
 - **$m \leq n \cdot (n - 1)$**
- If in-edges and out-edges are kept in separate adjacency lists
 - can perform listing of incoming edges and outgoing edges in time proportional to their size

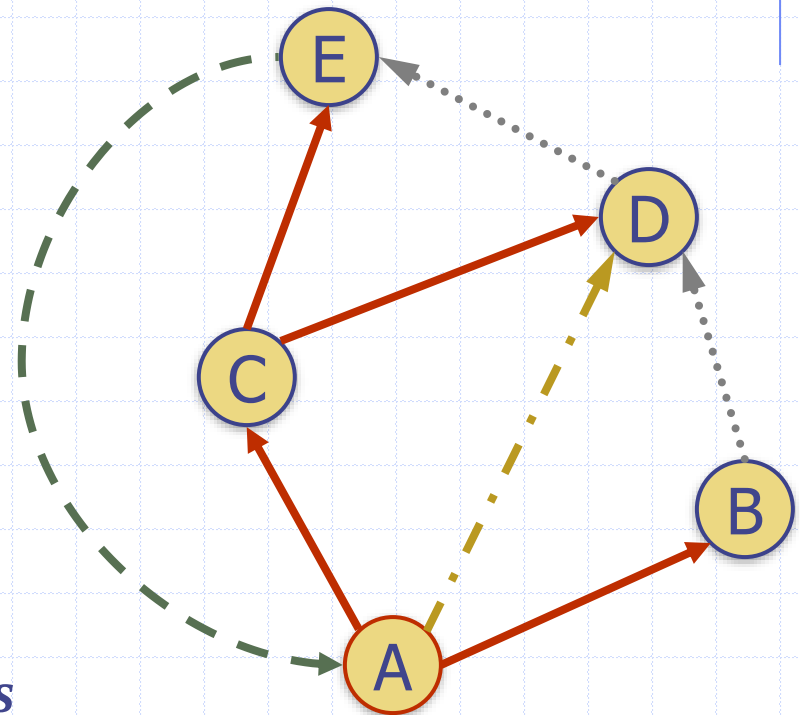
Digraph Application

- **Scheduling:** edge (a,b) means task a must be completed before b can be started



Directed Graph Traversal

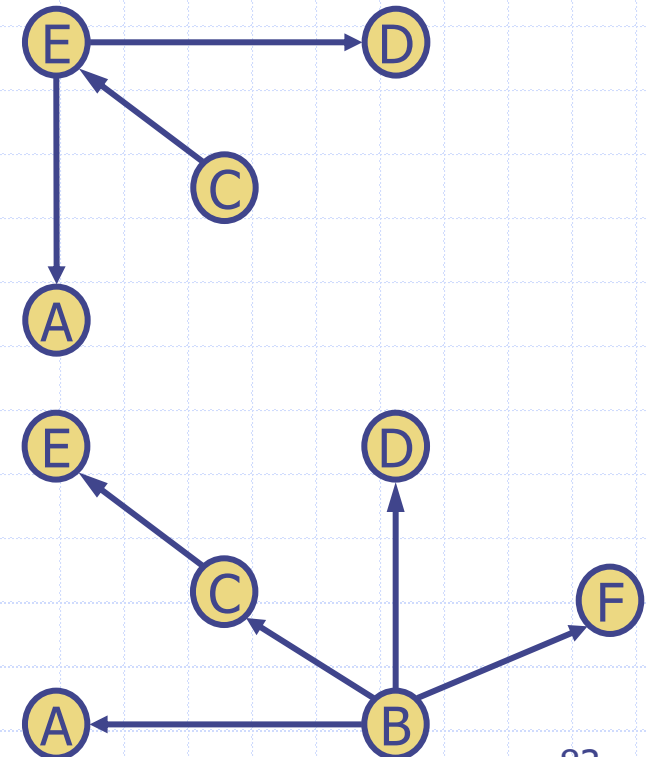
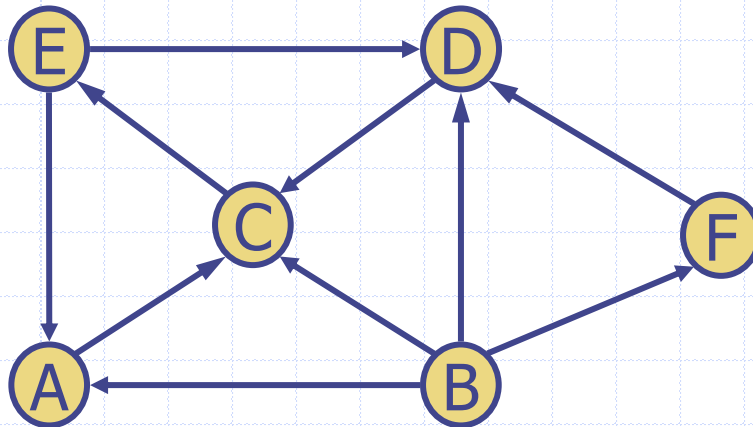
- ❑ Specialise (DFS & BFS) traversal algorithms for digraphs by traversing edges only along their direction
- ❑ Directed DFS algorithm has four types of edges
 - **discovery edges**
 - back edges
 - **forward edges**
 - cross edges
- ❑ Directed DFS starting at vertex s determines the vertices **reachable** from s

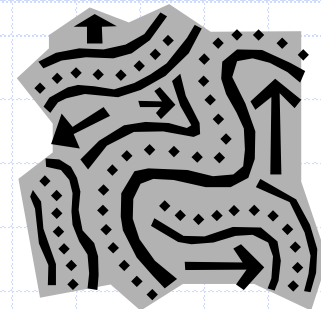


Reachability



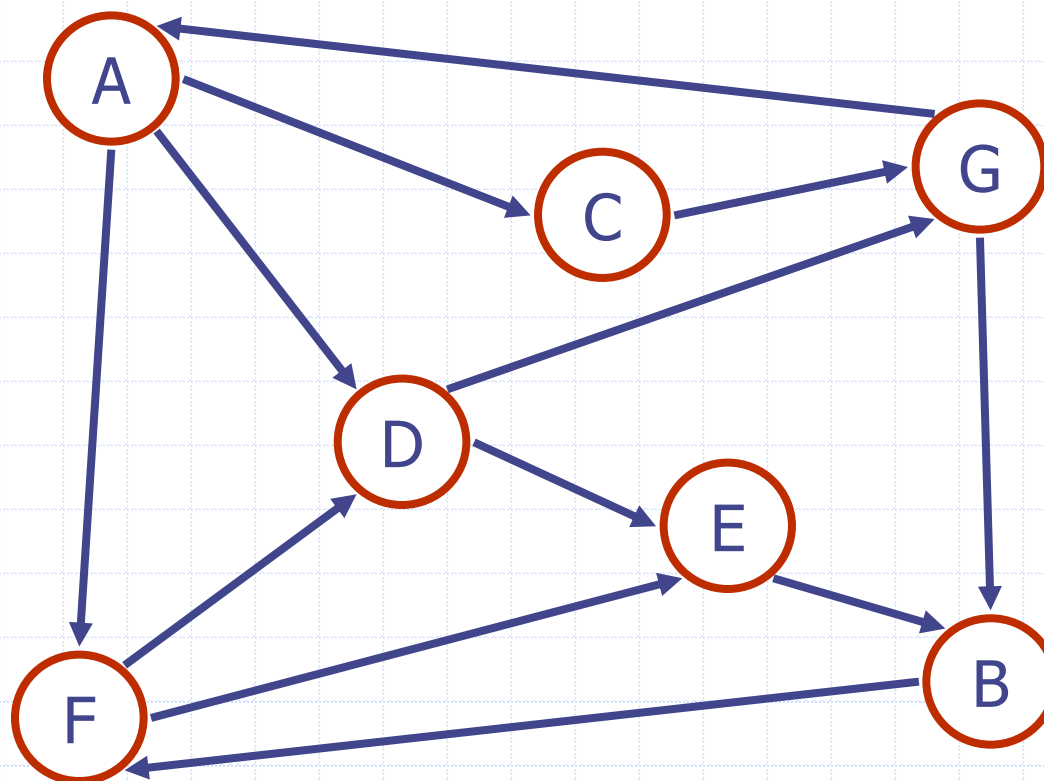
- DFS tree rooted at v
 - vertices reachable from v via directed paths





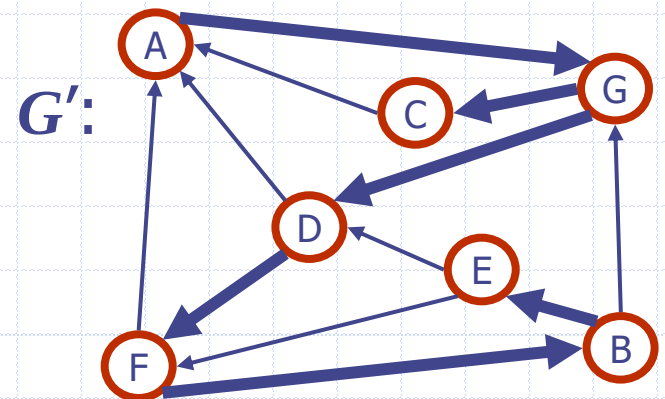
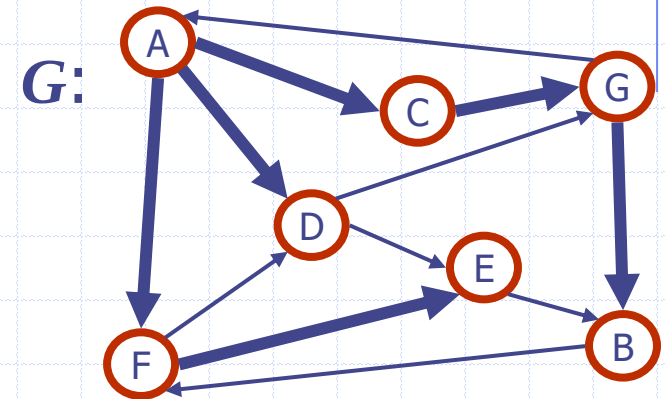
Strong Connectivity

- Each vertex can reach all other vertices



Strong Connectivity Algorithm

- ❑ Pick a vertex v in G
- ❑ Perform a DFS from v in G
 - if there's a w not visited, **false**
- ❑ Let G' be G with edges reversed
- ❑ Perform a DFS from v in G'
 - if there's a w not visited, **false**
 - else, **true**
- ❑ Running time: $O(n+m)$

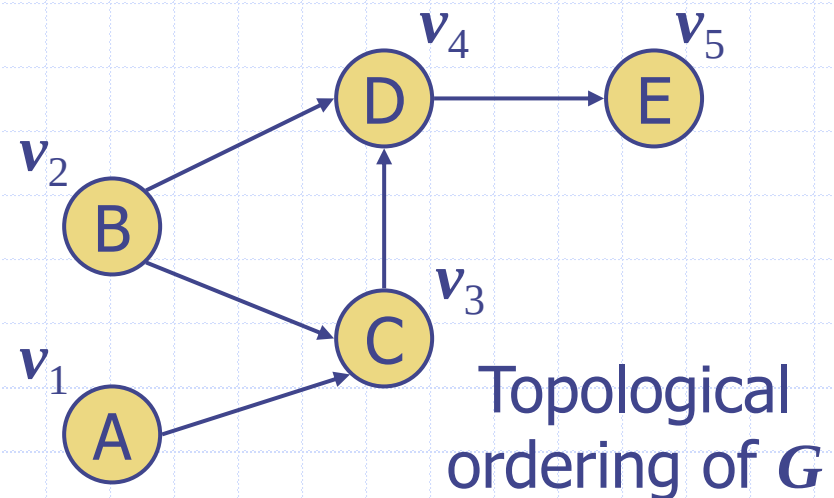
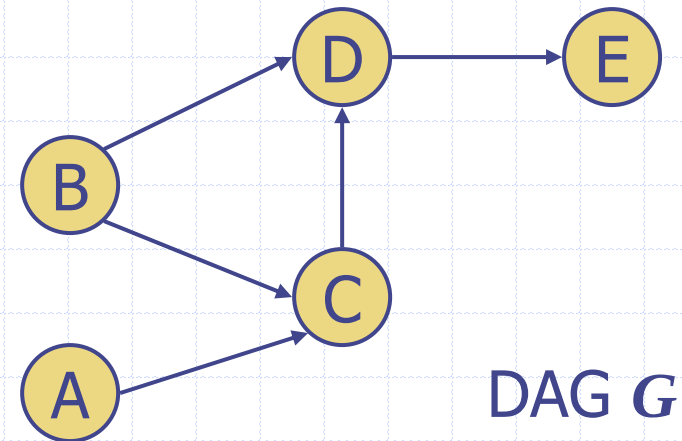


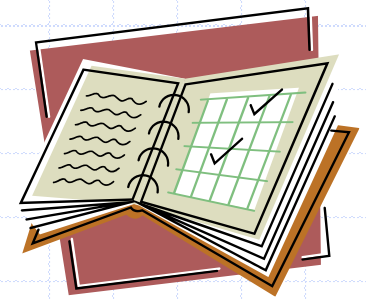
DAGs and Topological Ordering

- Directed acyclic graph (DAG)
 - digraph with no directed cycles
- Topological ordering of a digraph is a numbering $v_1, \dots, v_n$ of the vertices such that for every edge (v_i, v_j) , we have $i < j$
- Example: in a task scheduling digraph, a topological ordering of a task sequence that satisfies the precedence constraints

Theorem

A digraph has a topological ordering **if and only if** it is a DAG

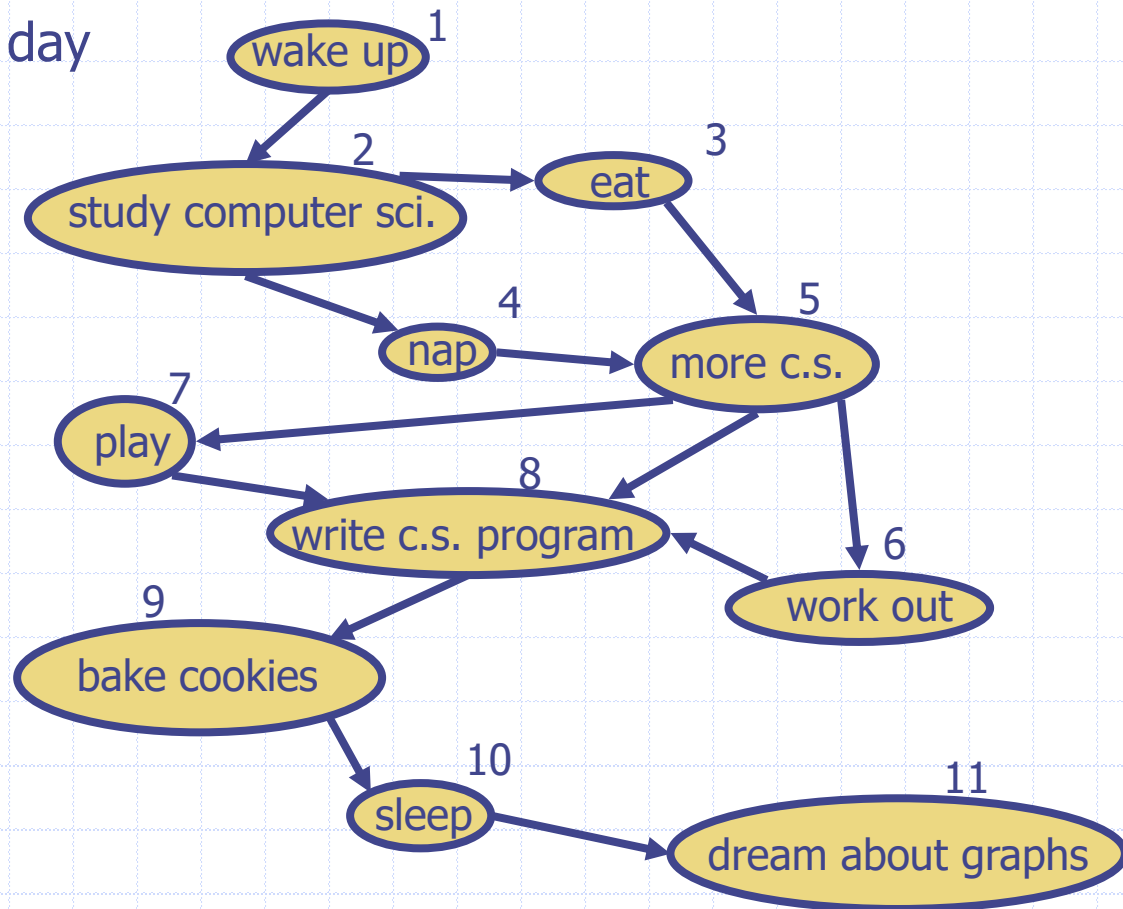




Topological Sorting

- Number vertices so that (u,v) in G implies $u < v$

A typical student day



Algorithm for Topological Sorting

Algorithm *TopologicalSort*(G):

$H \leftarrow G$ // Temporary copy of G

$n \leftarrow G.\text{numVertices}()$

while H is not empty **do**

 Let v be a vertex with no outgoing edges

 Label $v \leftarrow n$

$n \leftarrow n - 1$

 Remove v from H

□ Running time: $O(n + m)$

Implementation with DFS

- Simulate the algorithm with DFS
- $O(n+m)$ time

Algorithm *topologicalDFS(G)*

Input dag G

Output topological ordering of G

$n \leftarrow G.numVertices()$

for all $u \in G.vertices()$ **do**

Set u to be unvisited

for all $v \in G.vertices()$ **do**

if v has not been visited **then**
topologicalDFS(G, v)

Algorithm *topologicalDFS(G, v)*

Input graph G and a start vertex v of G

Output labeling of the vertices of G
in the connected component of v

Mark vertex v as visited

for all $e \in G.outgoingEdges(v)$ **do**

$w \leftarrow G.opposite(v, e)$

if w has not been visited **then**

{ e is a discovery edge }

topologicalDFS(G, w)

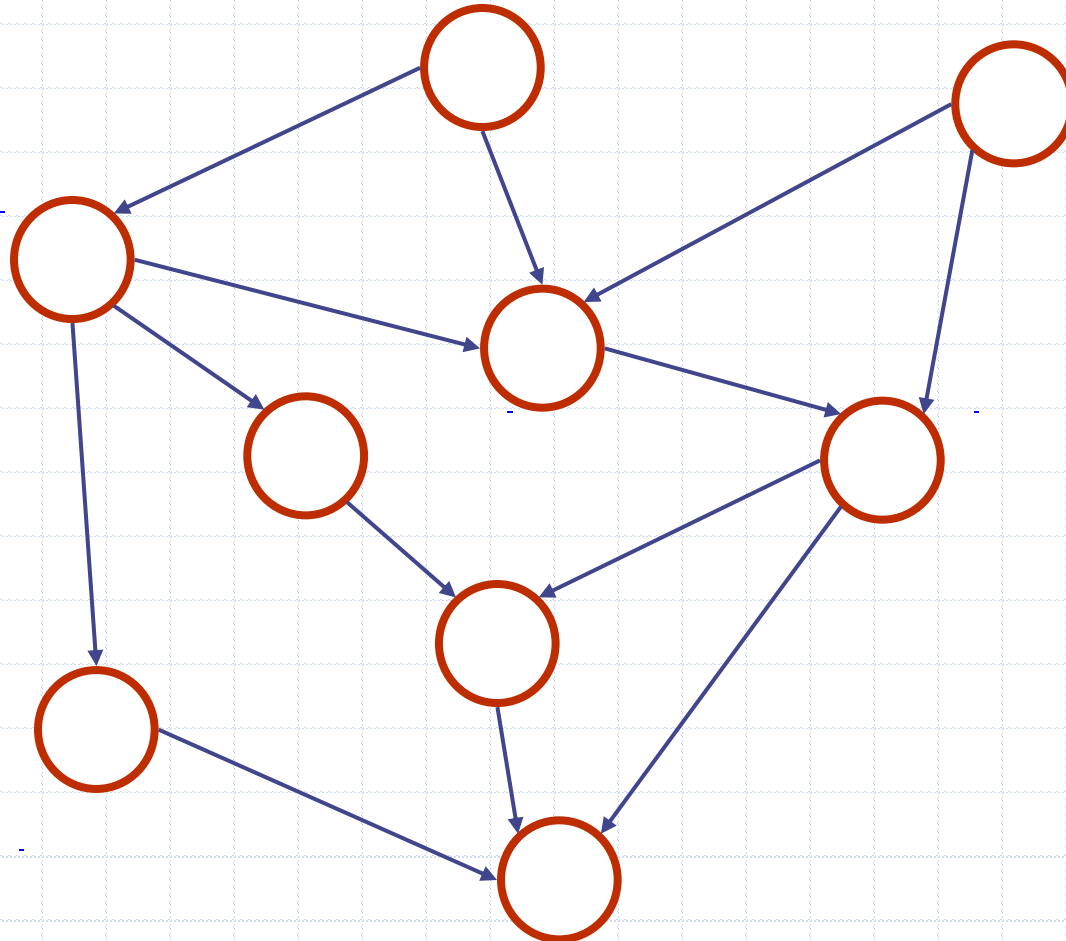
else

{ e is a forward or cross edge }

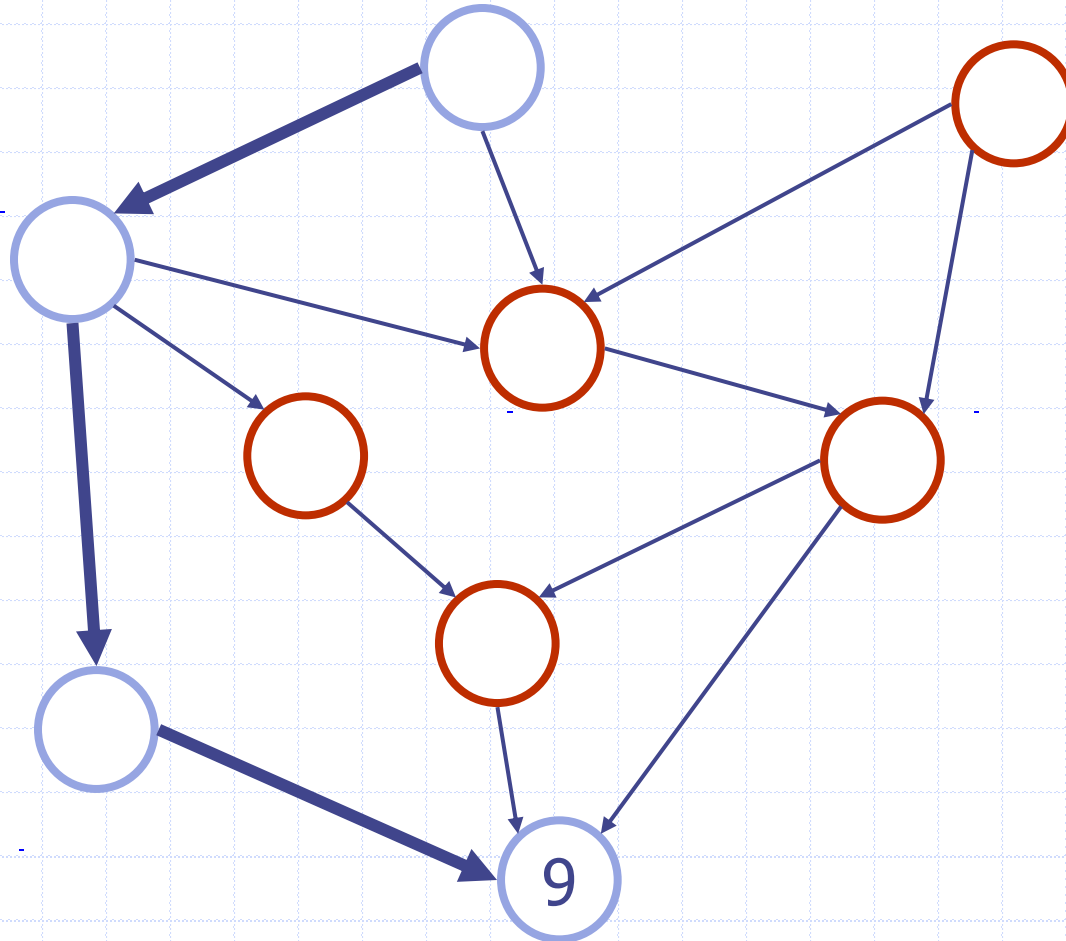
Label v with topological number n

$n \leftarrow n - 1$

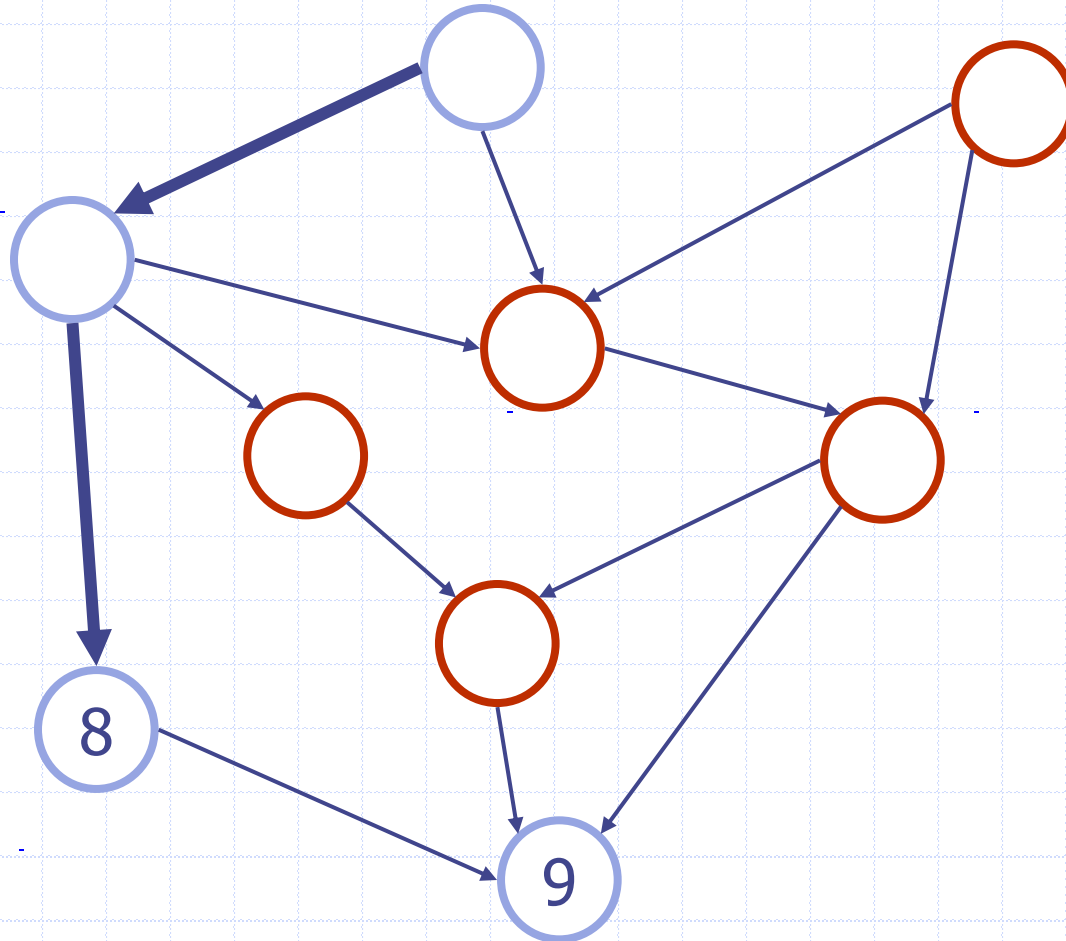
Topological Sorting Example

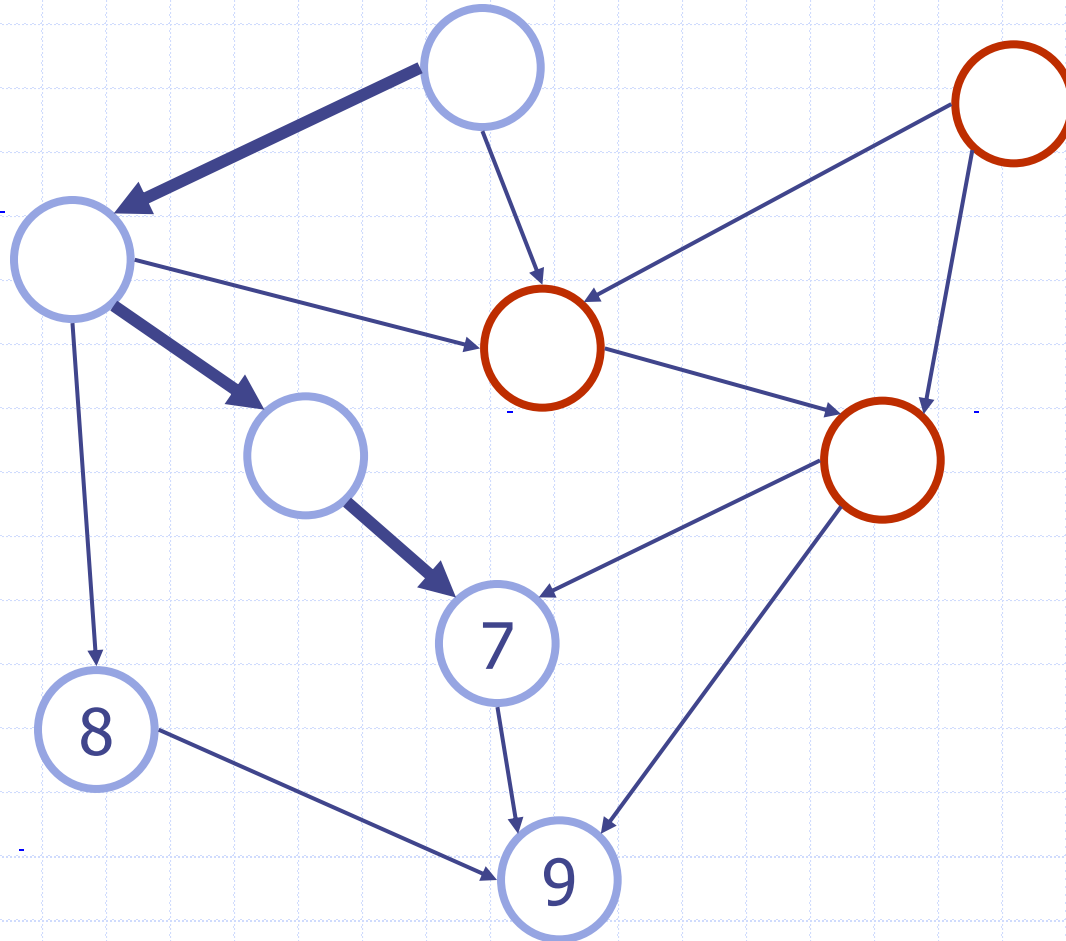


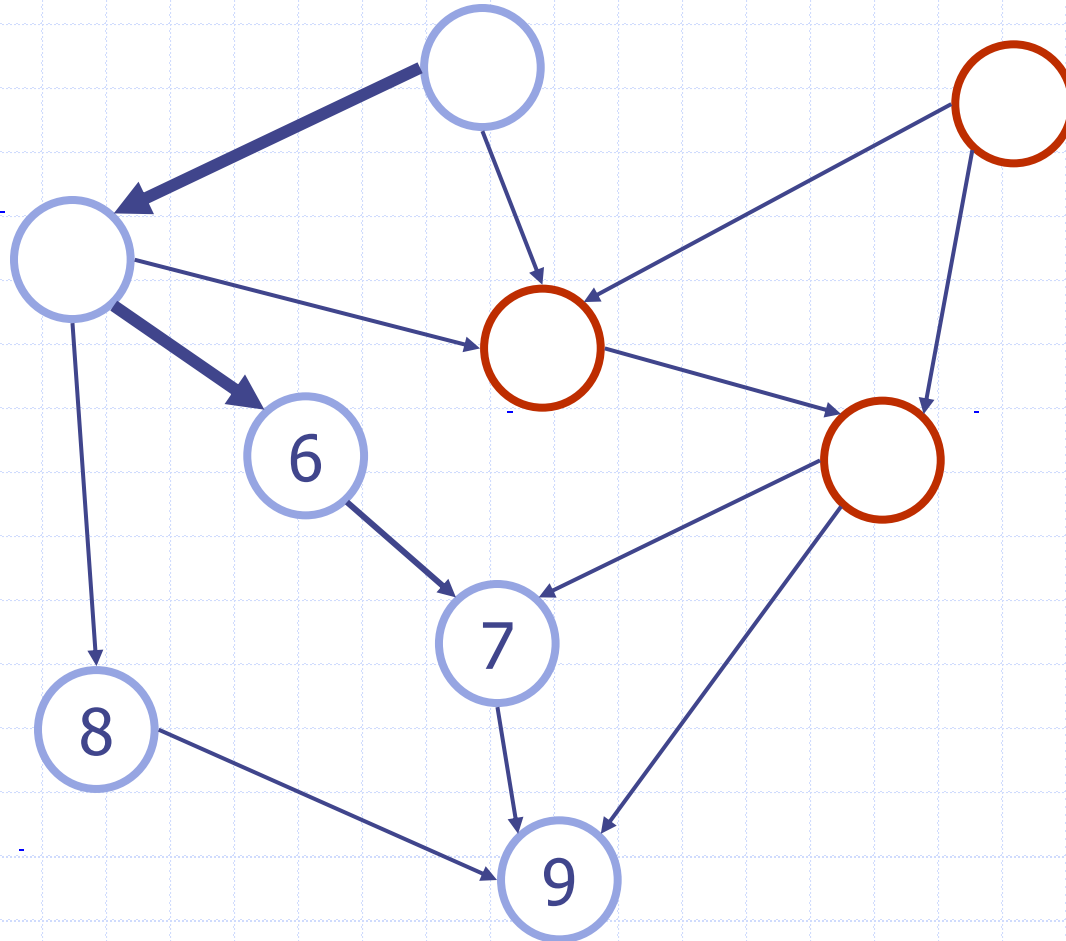
Topological Sorting Example



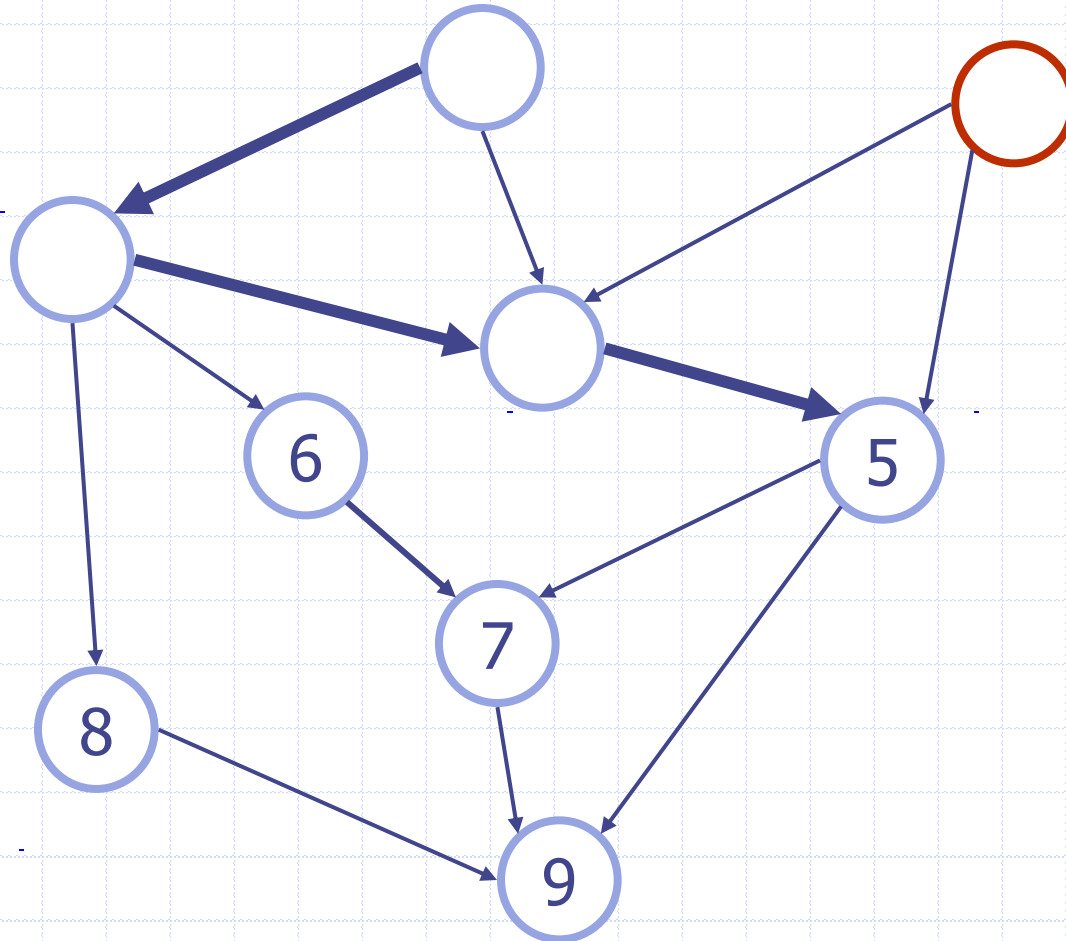
Topological Sorting Example



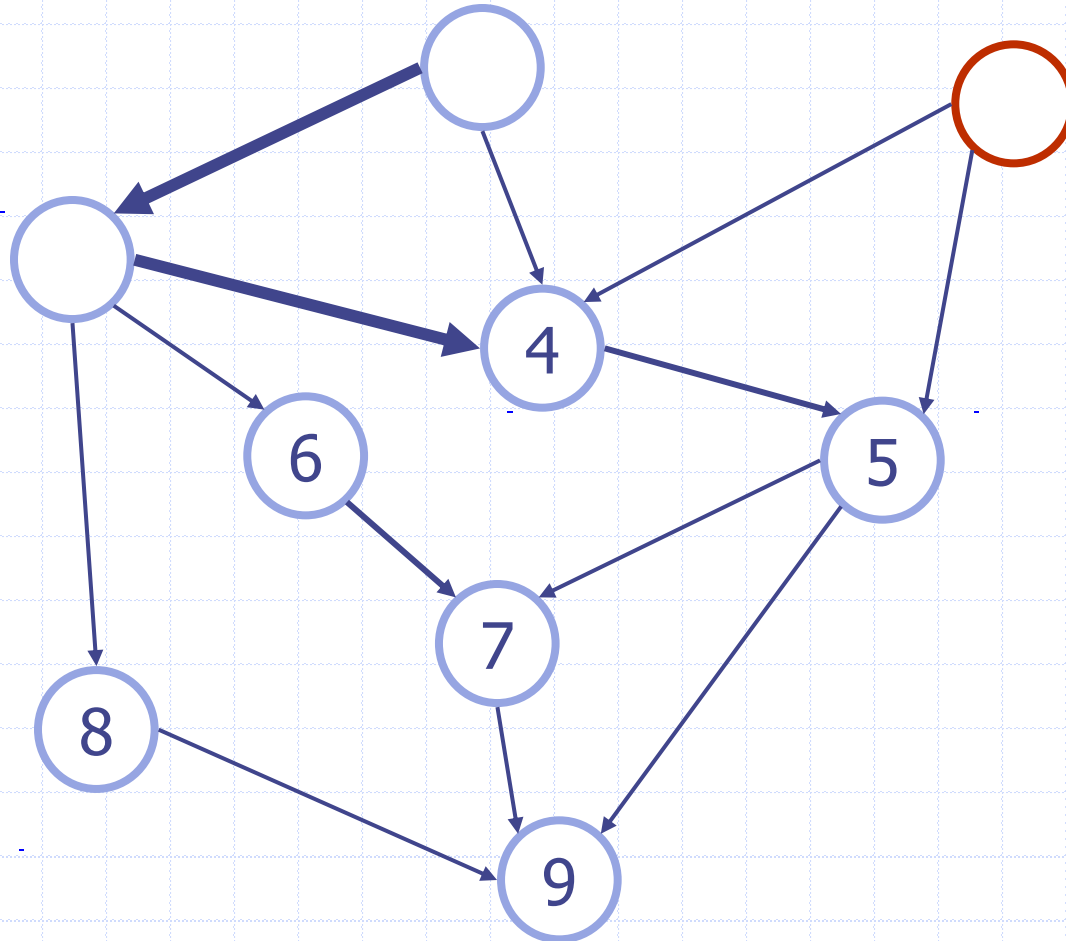




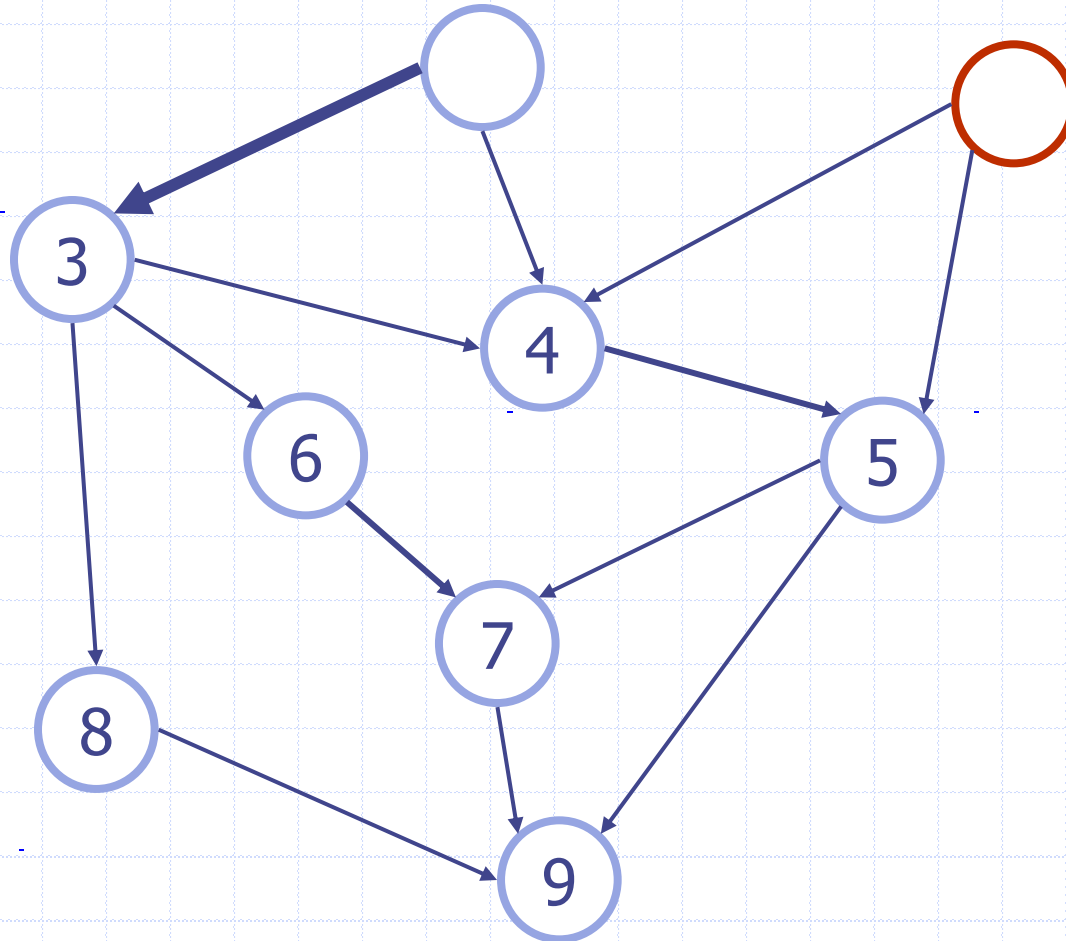
Topological Sorting Example



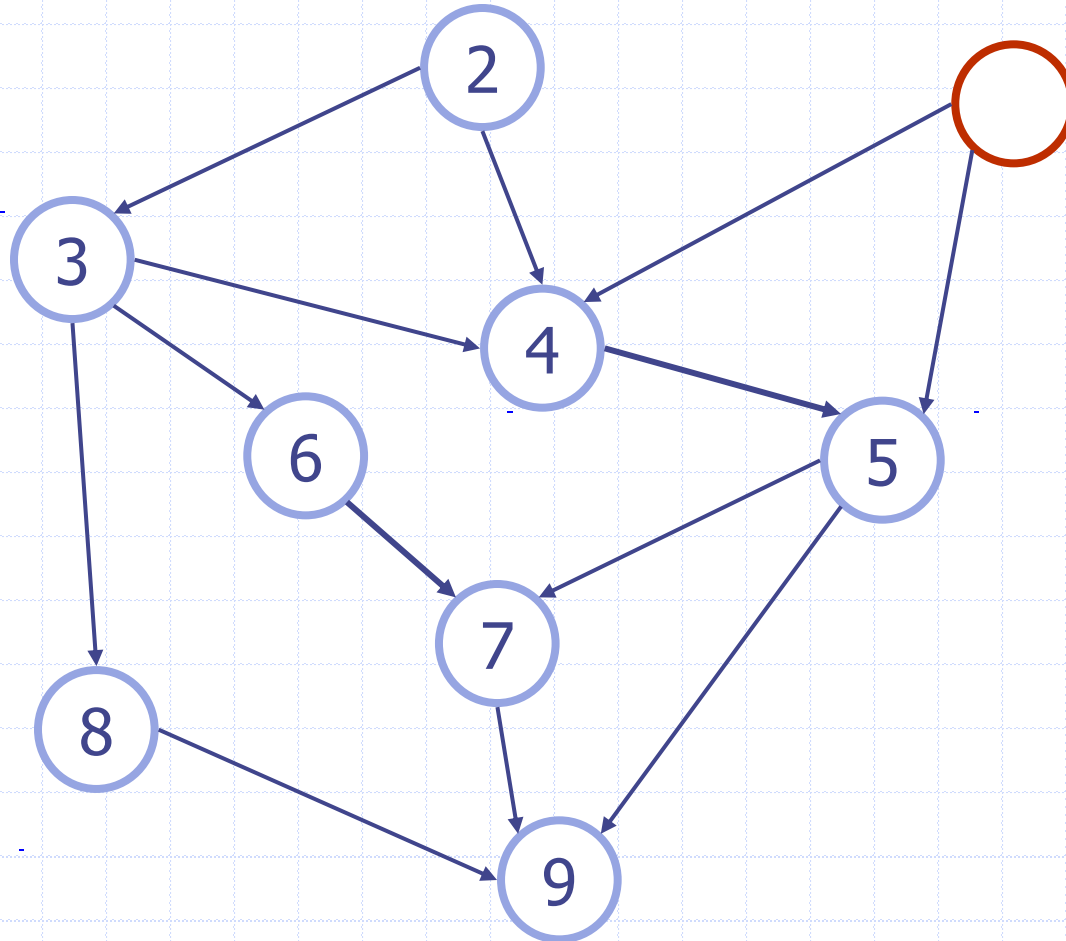
Topological Sorting Example



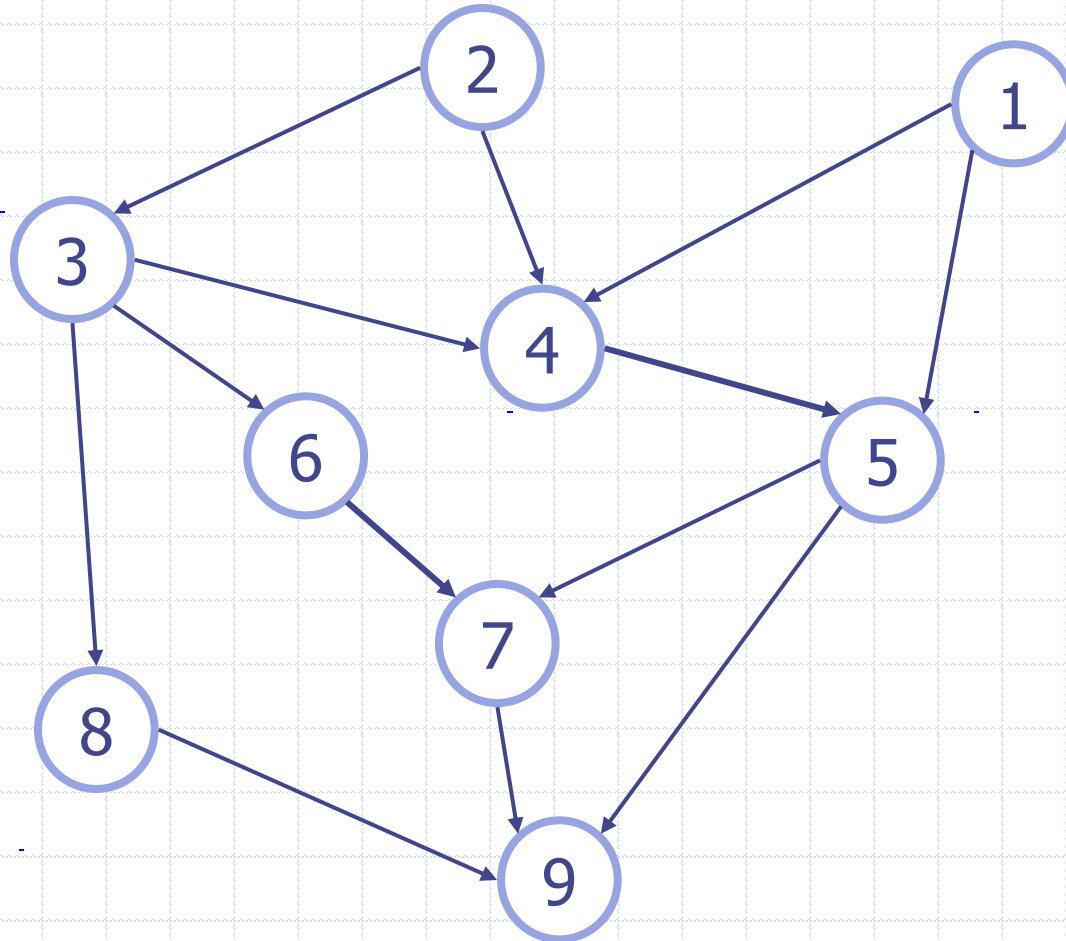
Topological Sorting Example



Topological Sorting Example



Topological Sorting Example



Further Reading

- Data Structures and Algorithms in Python
 - Chapter 14.1-14.5
- Introduction to Algorithms
 - Chapter 22